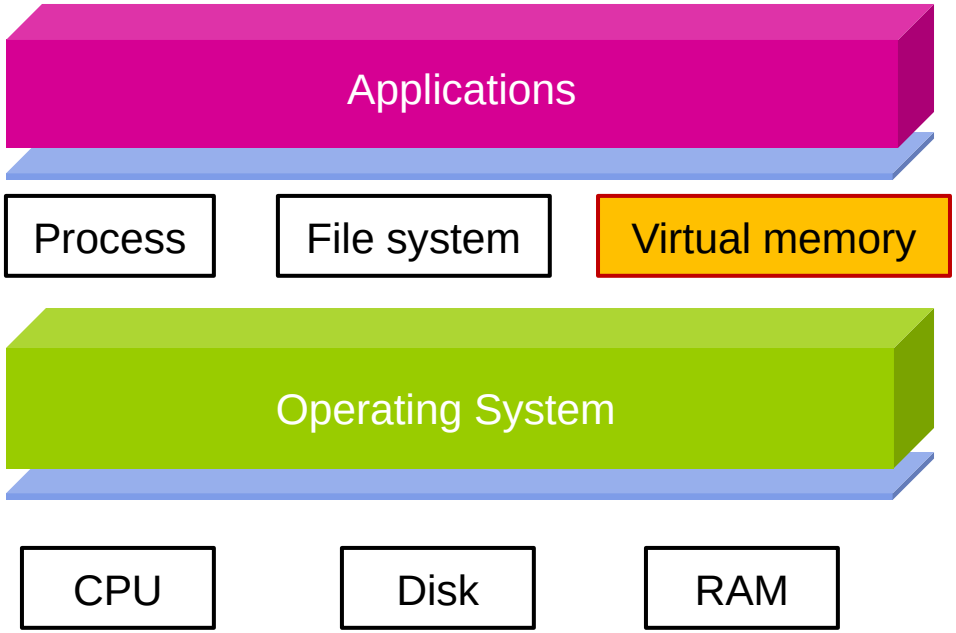
Virtual Memory

CS 202: Advanced Operating Systems

OS Abstractions



Memory Crisis in Early 1970s

- Situation
  - Memory: tiny, expensive
  - Programs: growing in size and complexity, multitasking
  - MM: managing memory as big, contiguous blocks

- Problems
  - Crashes from out-of-memory
  - No isolation
  - Moving code/data causes fragmentation
  - Painful due to manual and inflexible MM

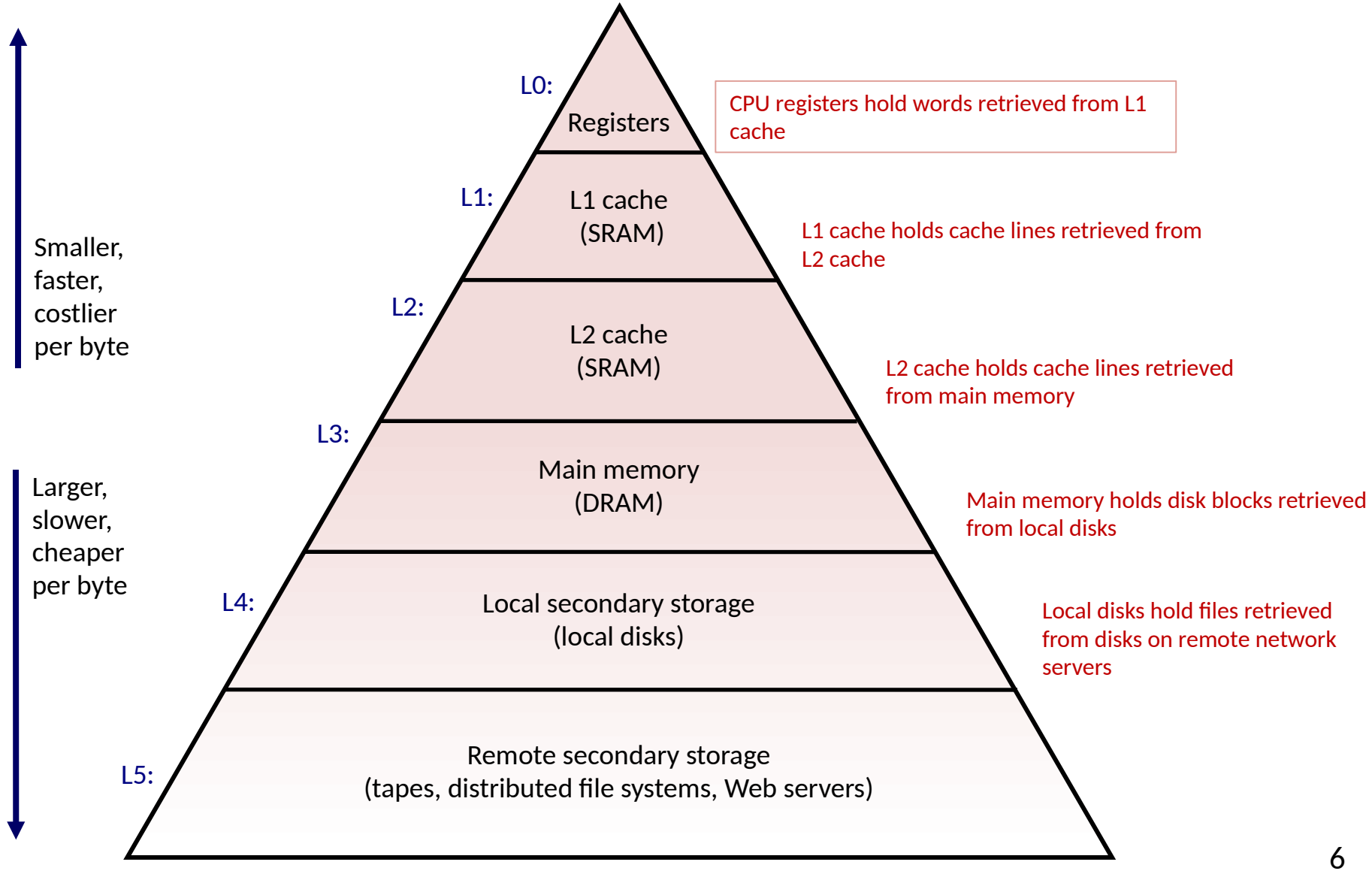
Today's lecture

- Problems using physical memory directly
- Virtual memory management via paging
- Problems with paging
  - Additional memory access that could be expensive
  - Huge page table
- Corresponding solutions to improve paging
  - TLB (cache for storing virtual to physical address translation)
  - Multi-level paging

Physical Memory

- Often called main memory (or DRAM: dynamic random-access memory)
  - A large array of words or bytes
  - The place where programs and information are kept when the processor is effectively utilizing them
  - Associated with the processor, so moving instructions and information into and out of the processor is extremely fast (compared to hard disks)
  - Volatile: main memory loses its data when a power interruption occurs

Example Memory Hierarchy



Memory Management

- In a multiprogramming computer, OS resides in a part of memory, and the rest is used by multiple processes
- The task of subdividing the memory among different processes is called Memory Management
- Manage operations between main memory and disk during process execution
- Goals: achieve efficient and secure utilization of memory

Problems with physical memory

- Size?
  - Crash if trying to access more than what we have
- Holes in address space?
  - Fragmentation
- Isolation?
  - Multiple programs may access the same address
- Motivation behind virtual memory
  - Indirection
  - Key: flexibility in how we use the physical memory
    - Map memory to disk (unlimited memory)
    - Fill holes in DRAM address space
    - Memory isolation among multiple programs

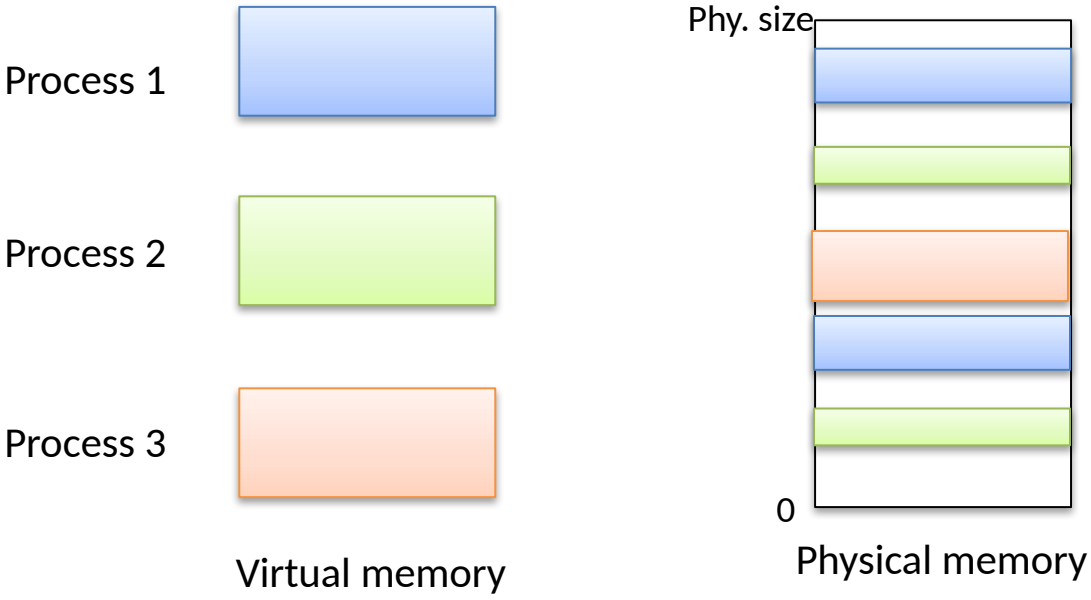
Solution: Virtual Memory via Paging

- What would you do?
  - Contiguous necessary?
  - What if we could break it into fixed-size blocks, place them anywhere, and let programs think they had all the memory they wanted?

- **VM via paging: break memory into fixed-size blocks, map them freely!**
  - Simple, uniform page-based memory
  - No fragmentation
  - Enable virtual memory: private address spaces
  - Flexible mapping (virtual → physical)
  - Allowed overcommitment: on-demand paging
  - Foundation for modern OSes

Virtual vs. Physical Memory

- Physical memory: the actual memory installed in the computer
- Virtual memory: logical memory space owned by each process
  - Virtual memory space can be much larger than the physical memory space
  - Only part of the program needs to be in physical memory for execution



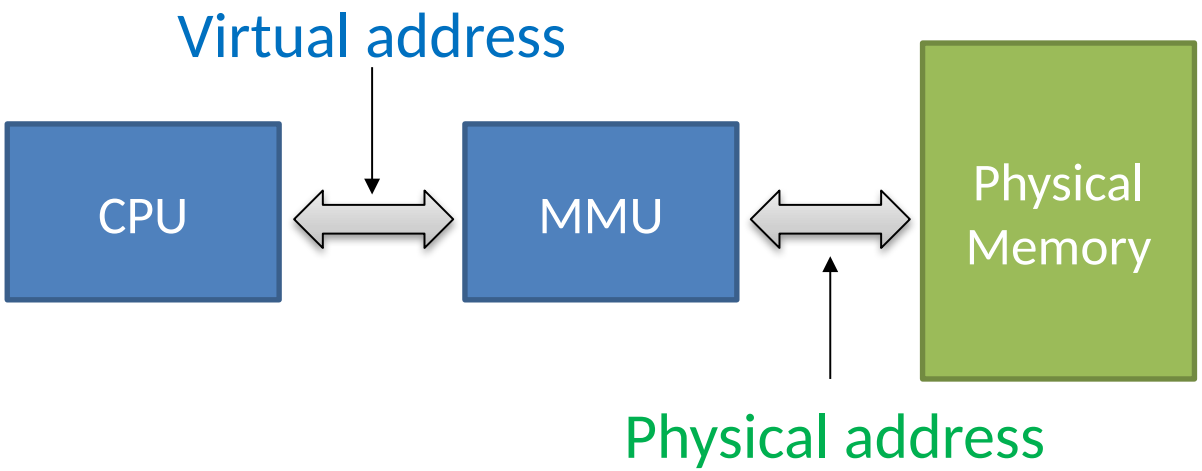
Virtual Address

- Processes use virtual (logical) addresses
  - Make it easier to manage memory of multiple processes
  - Virtual addresses are independent of the actual physical location of data referenced
  - Instructions executed by the CPU issue virtual addresses
  - Virtual address are translated by hardware into physical addresses (with help from OS)

- Mechanisms for virtual ↔ physical address translation
  - Hardware and OS support: MMU, Paging, and Page tables

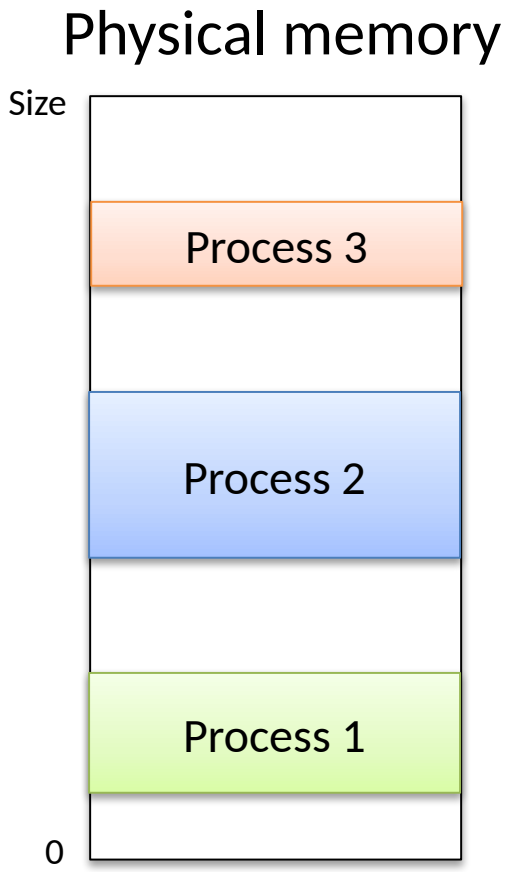
Virtual Address

*MMU: Memory Management Unit



- MMU: Translates virtual address to physical address dynamically at every reference
  - Many ways to do this translation…
  - Need hardware support and OS management algorithms
- Requirements
  - Protection – restrict which addresses processes can use
  - Fast translation – lookups need to be fast
  - Fast change – updating memory hardware on context switch

External Fragmentation

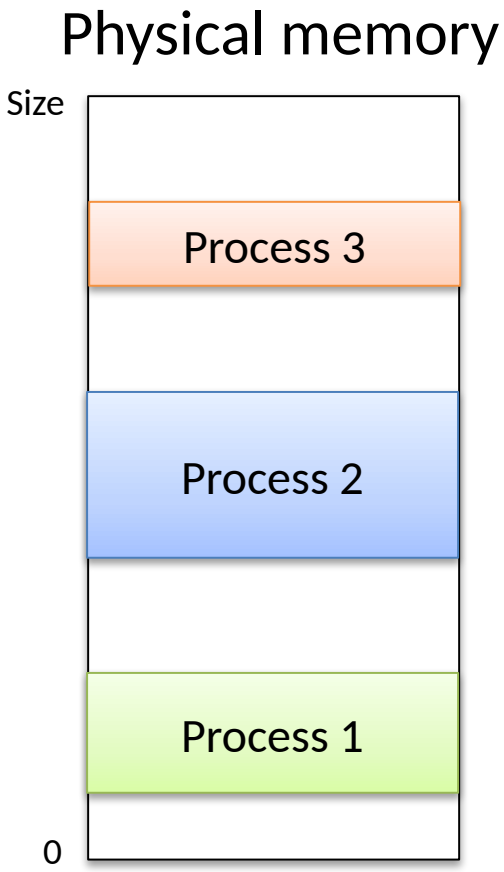


External Fragmentation

New process

Process 4

Not enough contiguous physical space

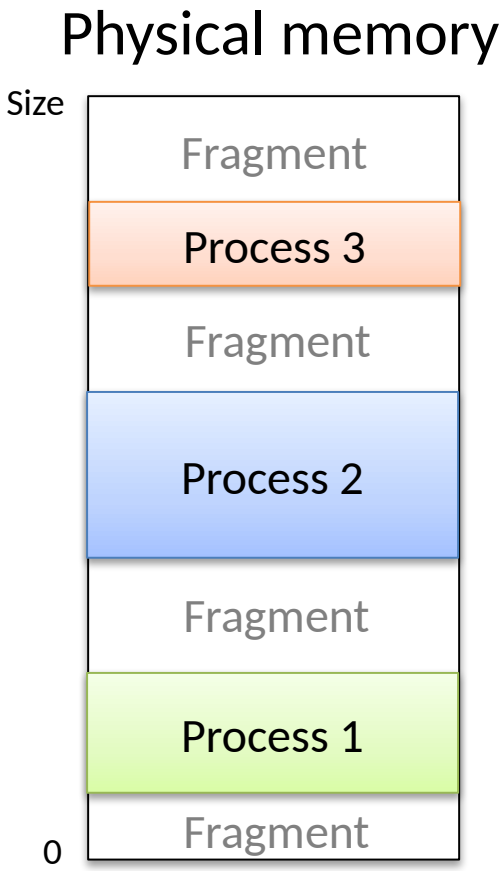


External Fragmentation

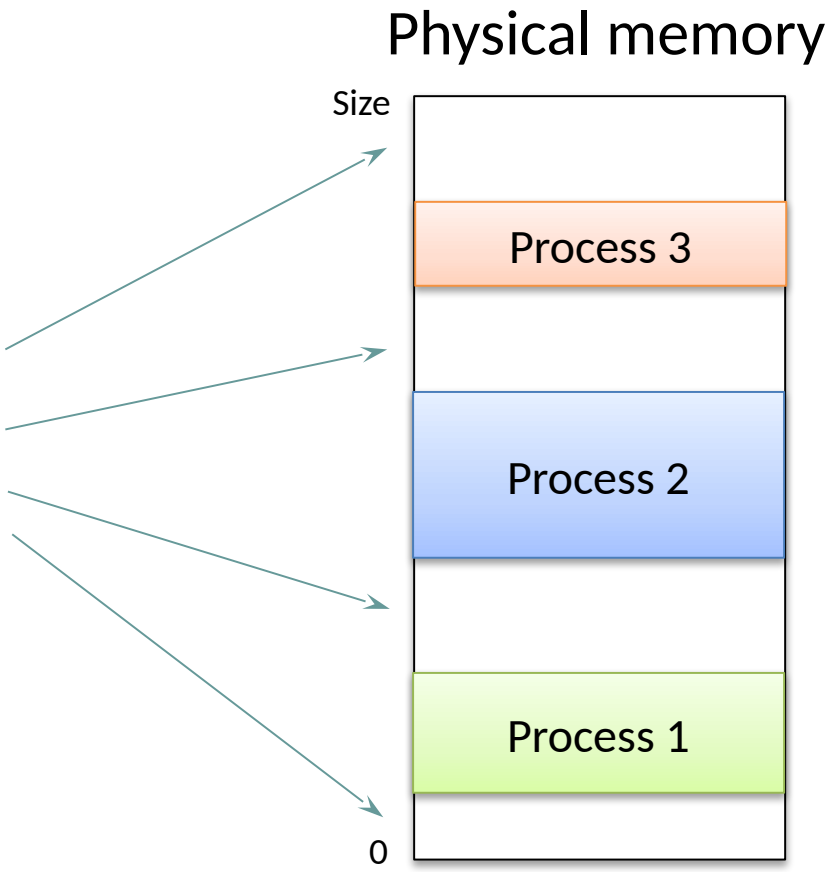
New process

Process 4

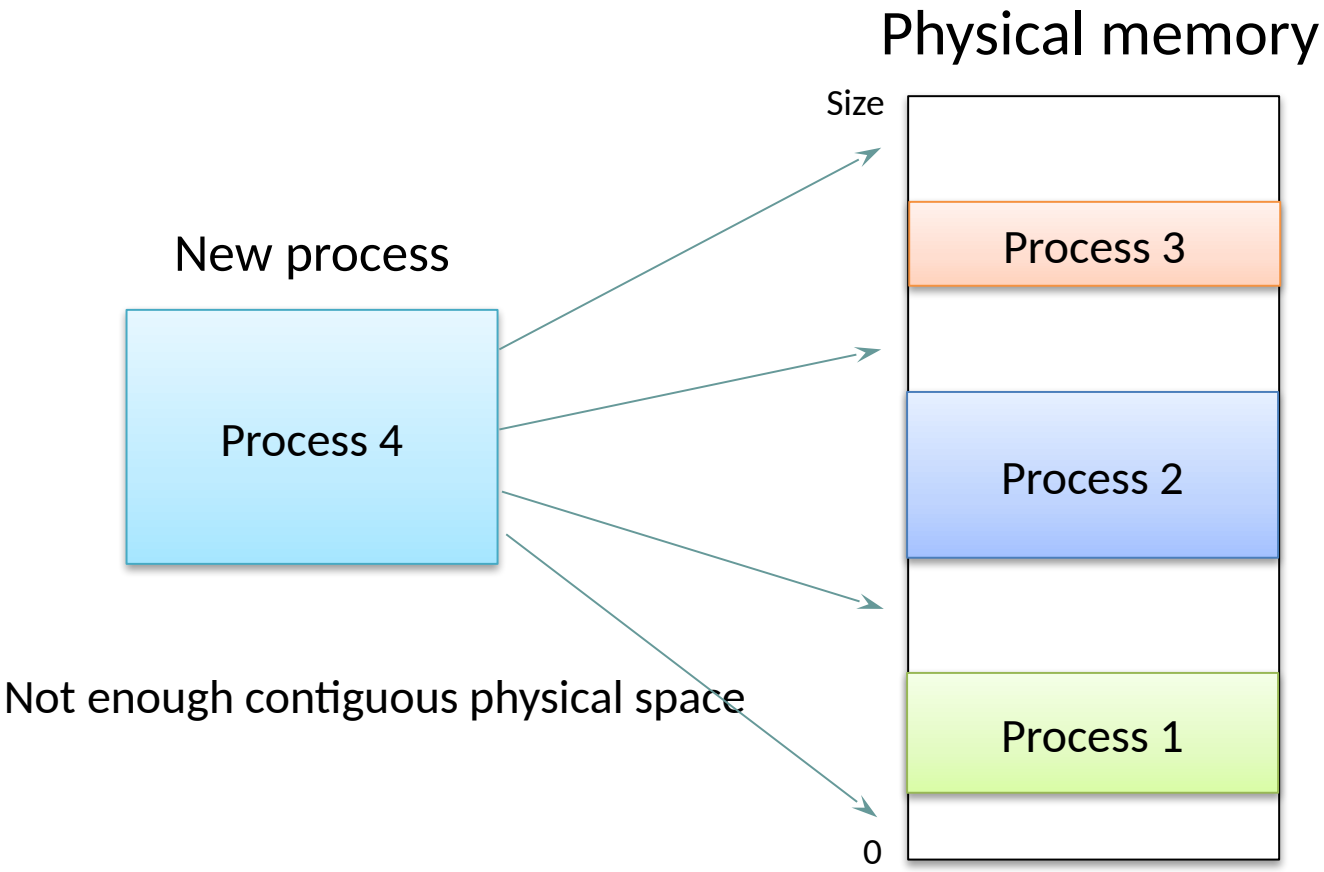
Not enough contiguous physical space



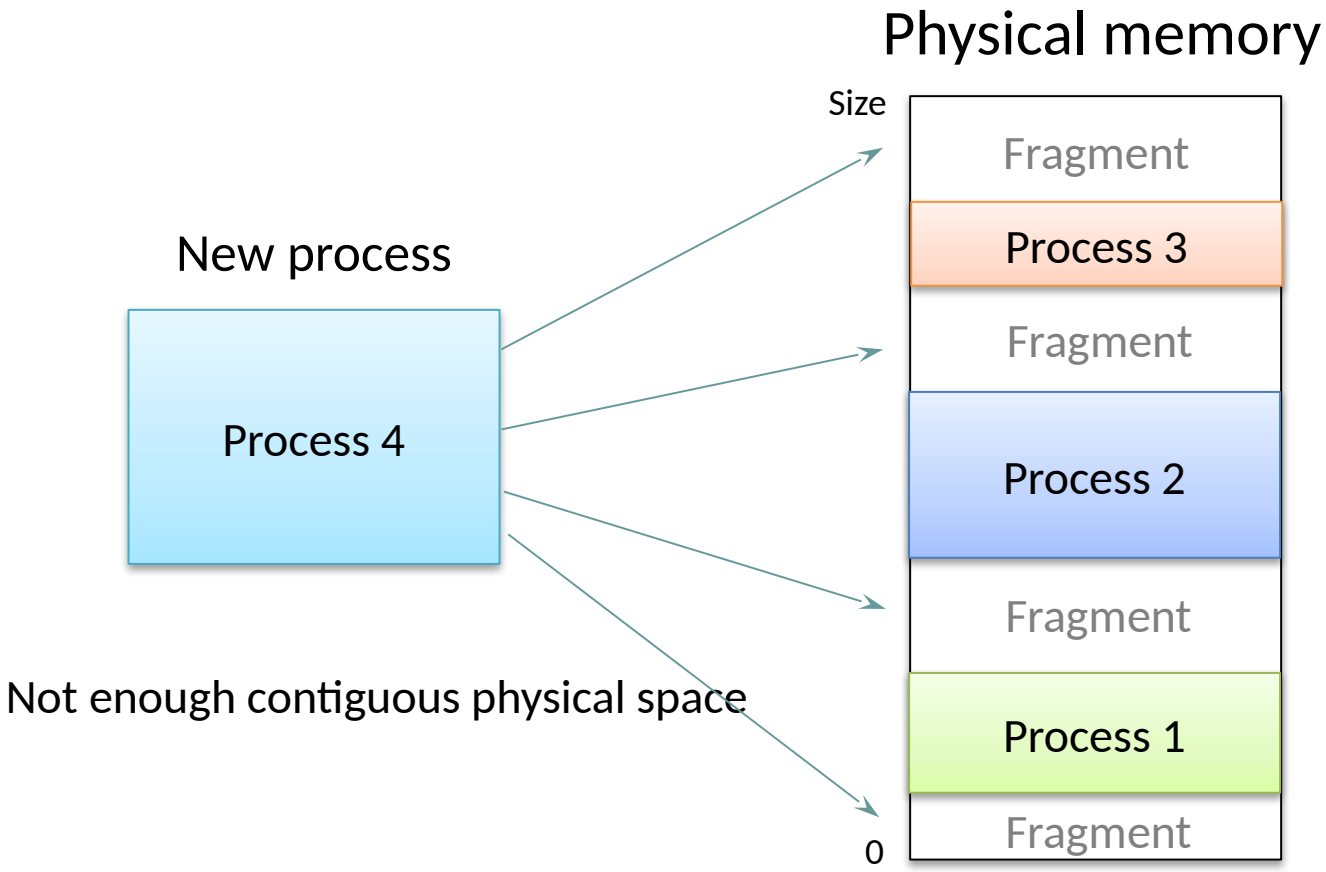
External Fragmentation



External Fragmentation



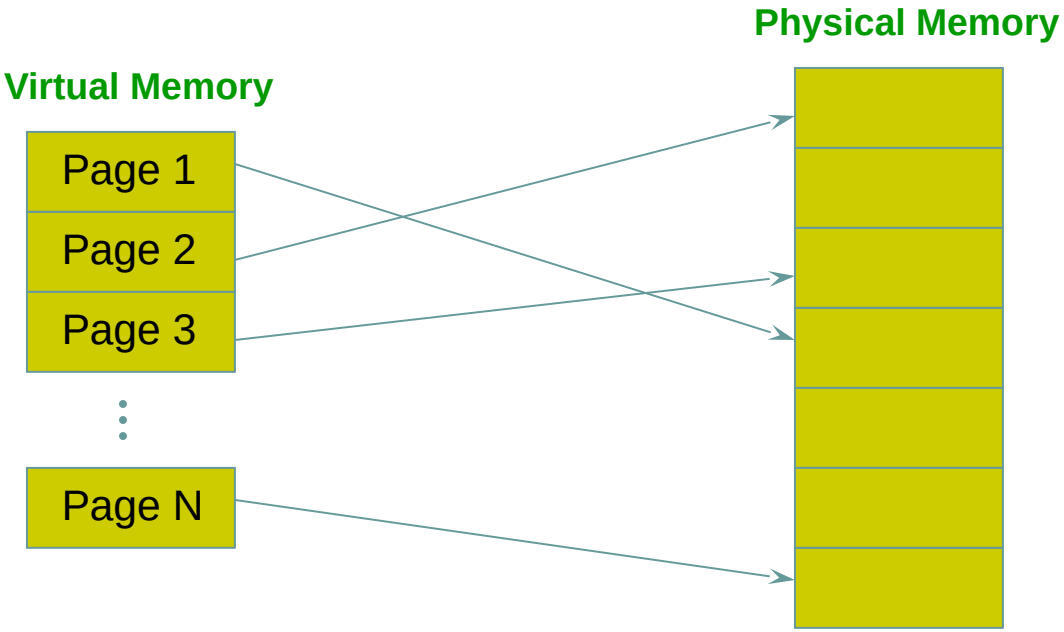
External Fragmentation



Paging

- Main Idea: Split address space into equal sized units called pages
  - Each can go anywhere!

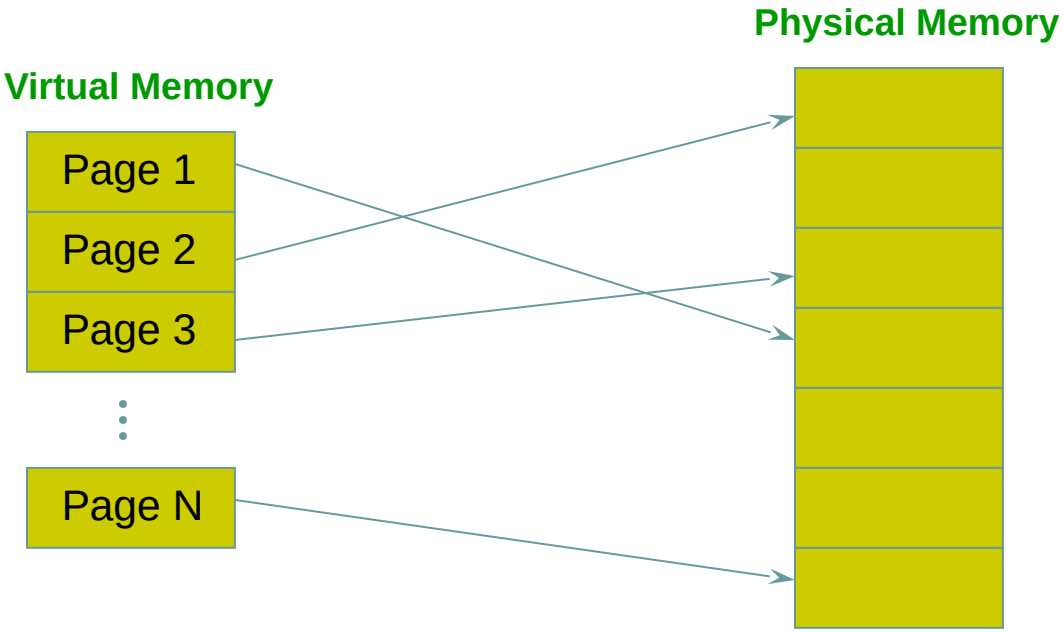
e.g., page size of 4KB



Paging

- Main Idea: Split address space into equal sized units called pages
  - Each can go anywhere!

e.g., page size of 4KB

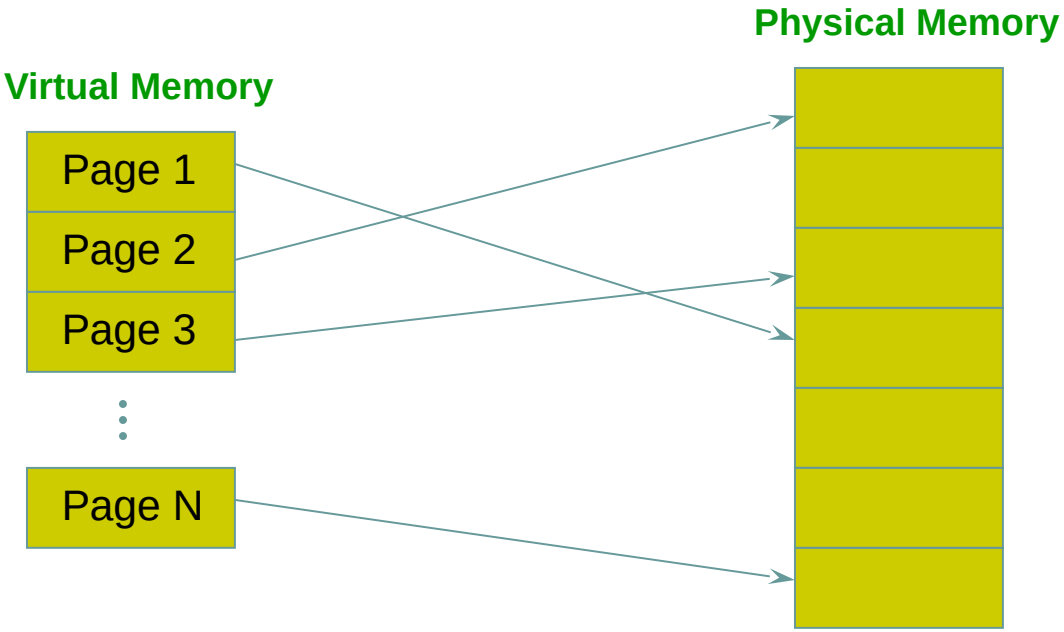


Paging solves the external fragmentation problem by dividing memory into fixed sized blocks that can placed anywhere in physical memory

Paging

- Main Idea: Split address space into equal sized units called pages
  - Each can go anywhere!

e.g., page size of 4KB

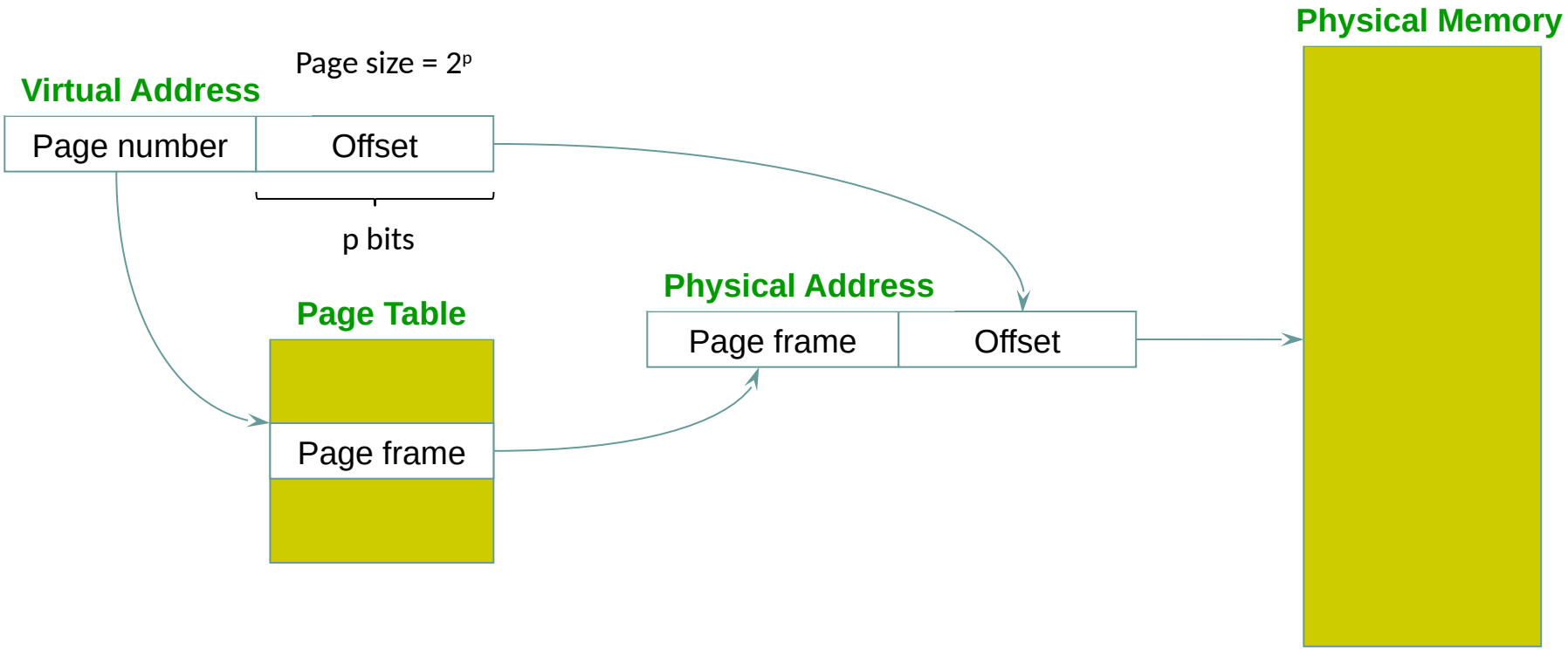


Paging solves the external fragmentation problem by dividing memory into fixed sized blocks that can placed anywhere in physical memory

But need to keep track of where things are!

Page Lookups

- *Page table:* an array of page table entries (PTEs) that maps virtual pages to physical pages
  - Per-process kernel data structure (part of process's state) (why? Threads?)

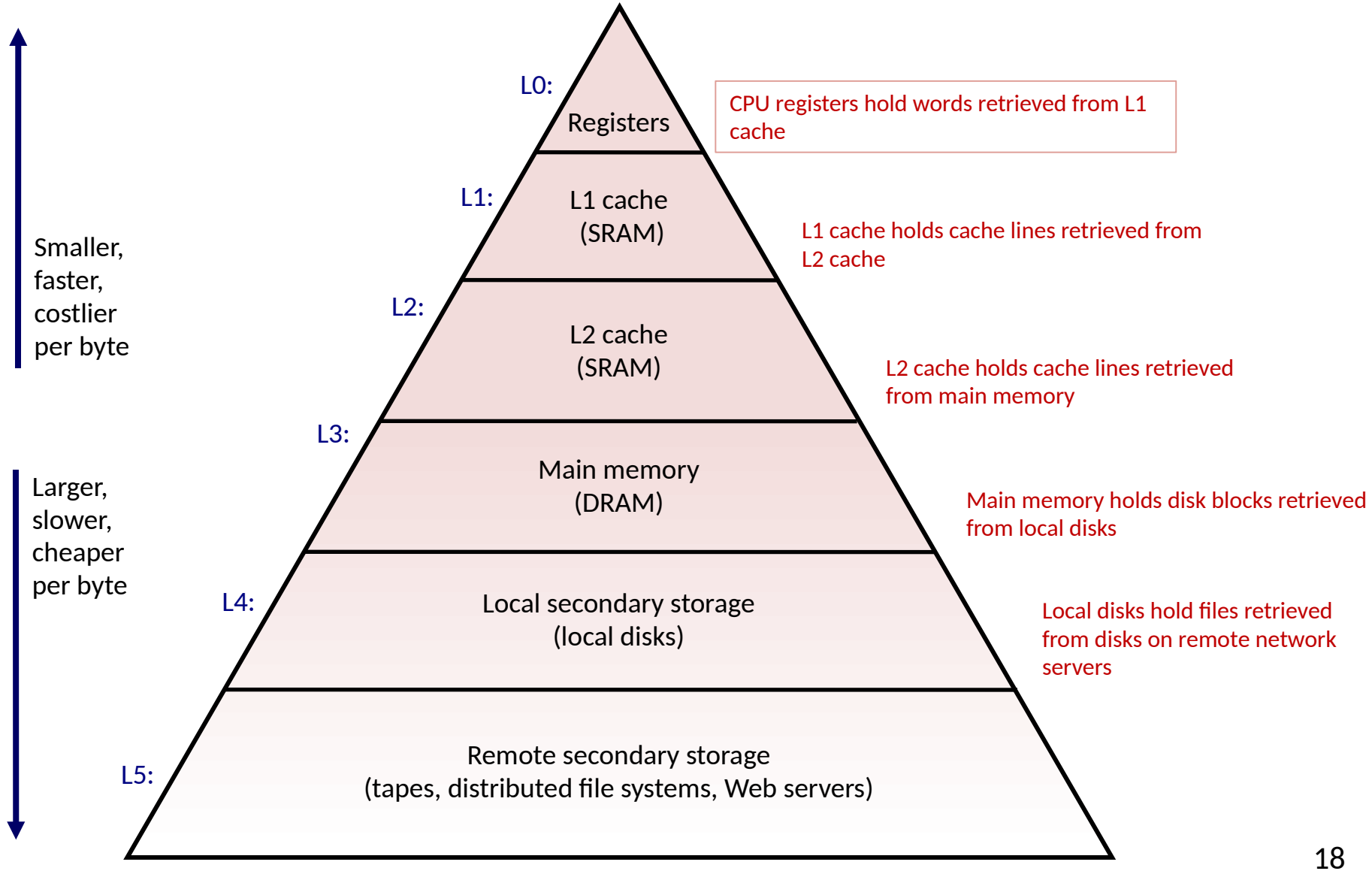


Why Virtual Memory (VM)?

- Simplifies memory management
  - Each process gets the same uniform linear address space

- With VM, we can:
  - Use DRAM as a cache for the parts of a virtual address space
  - Protect against illegal memory access
  - Share data among processes efficiently and safely

Example Memory Hierarchy



Memory hierarchy

- Cache: A smaller, faster storage device that acts as a staging area for a subset of the data in a larger, slower device.

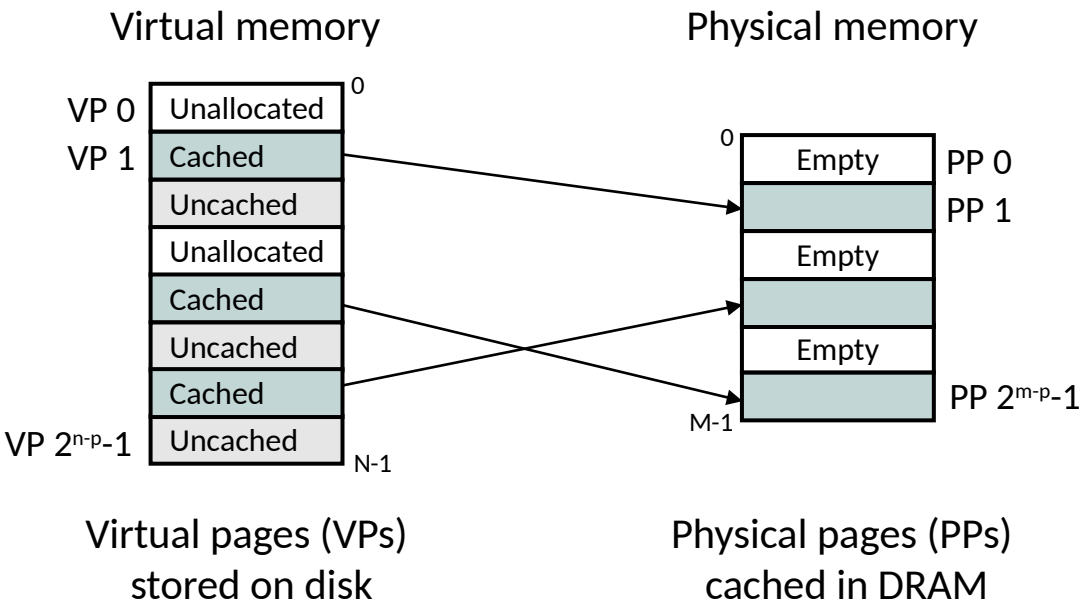
- Fundamental idea of a memory hierarchy:
  - For each layer, a faster, smaller device caches a larger, slower device

- Why do memory hierarchies work?
  - Because of locality!
    - Hit fast memory much more frequently even though its smaller
  - Thus, the storage at level k+1 can be slower (but larger and cheaper!)

- *Big Idea:* The memory hierarchy creates a large pool of storage that costs as much as the cheap storage near the bottom, but that serves data to programs at the rate of the fast storage near the top.

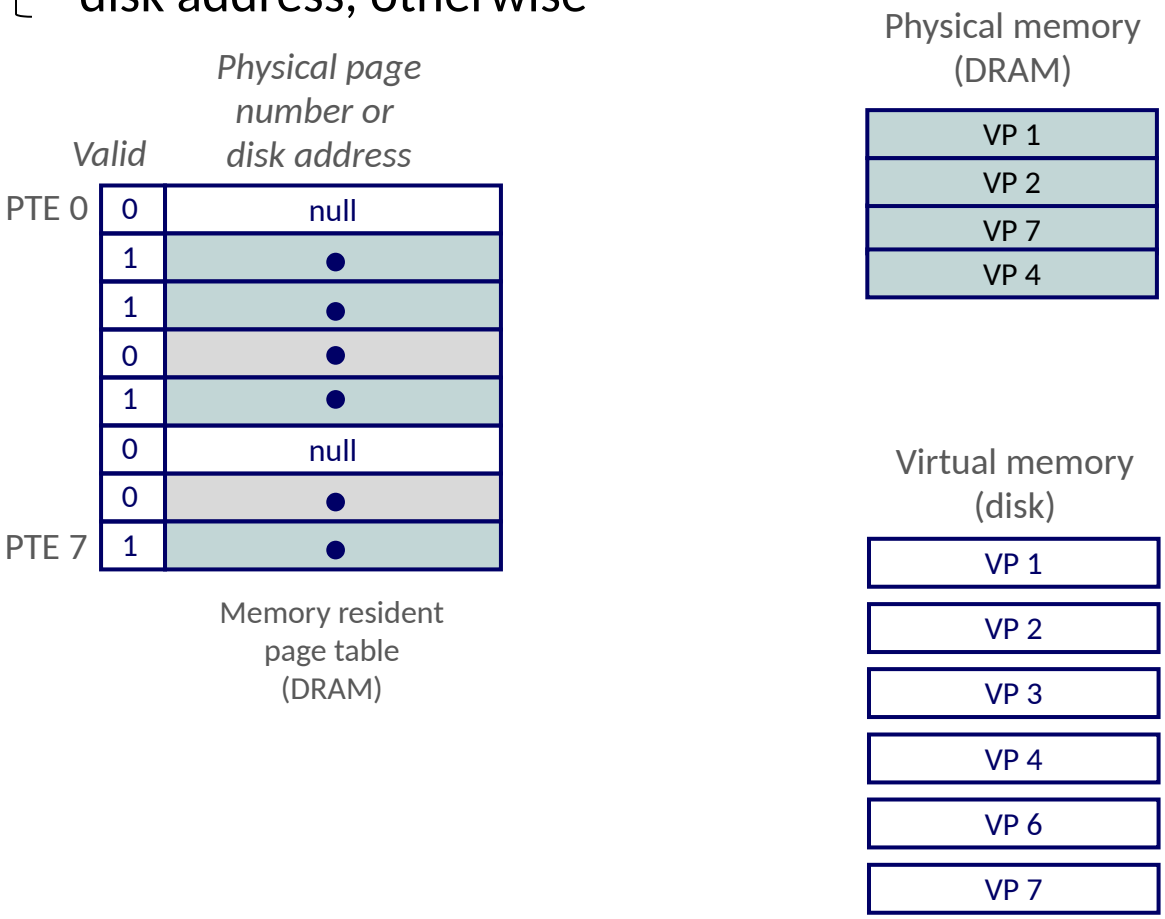
VM as a tool for treating DRAM as cache for disk

- *Virtual memory* mapped to *N* contiguous bytes stored on disk
- The contents of the array on disk are cached in *physical memory*
  - *DRAM as a cache for disk*
  - These cache blocks are called *pages* (size is P = 2^p bytes)



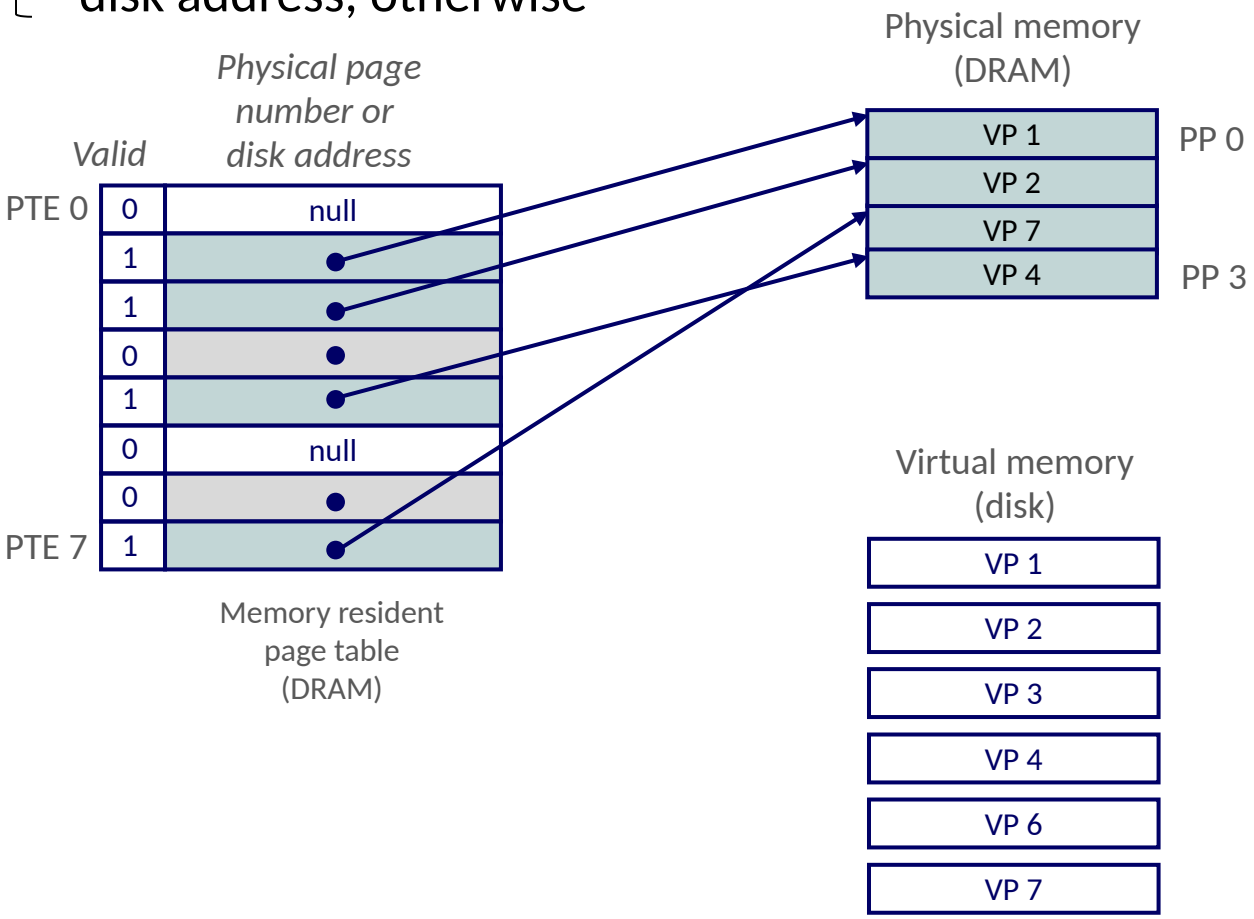
Page Tables

- *Page table:* page table entries (PTEs) map virtual to physical pages
  - PTE: - physical page number, if page is cached (resident) in DRAM
    - disk address, otherwise



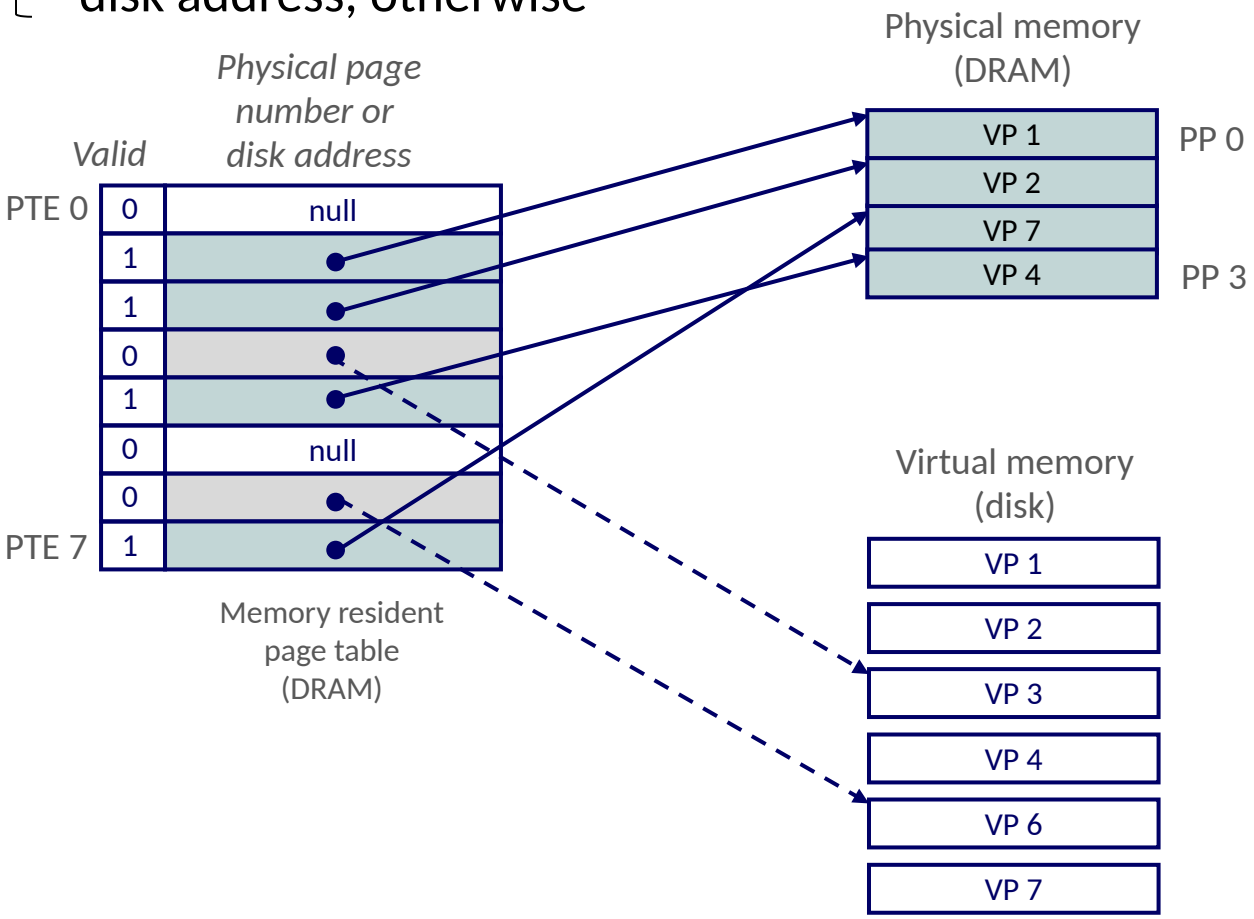
Page Tables

- *Page table:* page table entries (PTEs) map virtual to physical pages
  - PTE: { - physical page number, if page is cached (resident) in DRAM
    - disk address, otherwise



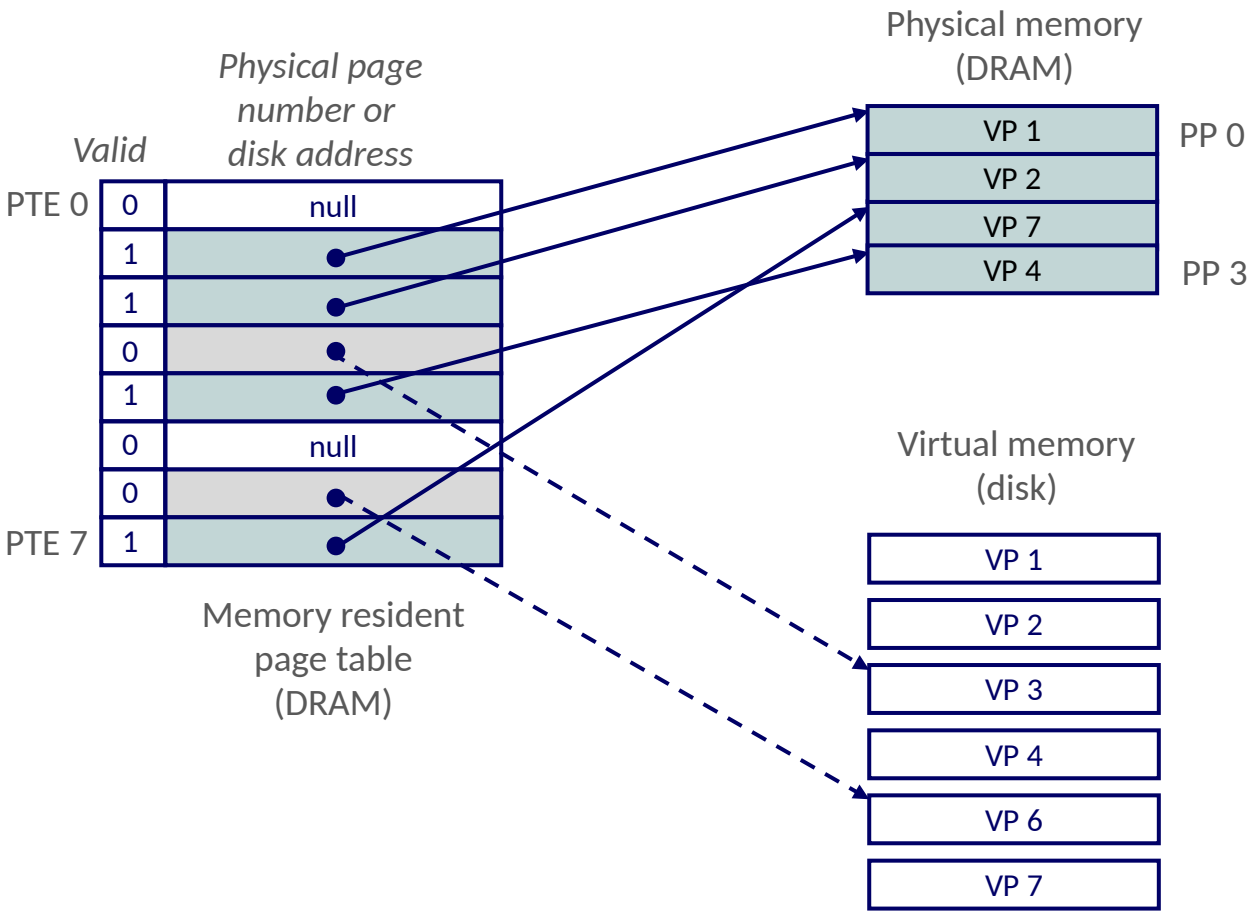
Page Tables

- *Page table:* page table entries (PTEs) map virtual to physical pages
  - PTE: { - physical page number, if page is cached (resident) in DRAM
    - disk address, otherwise



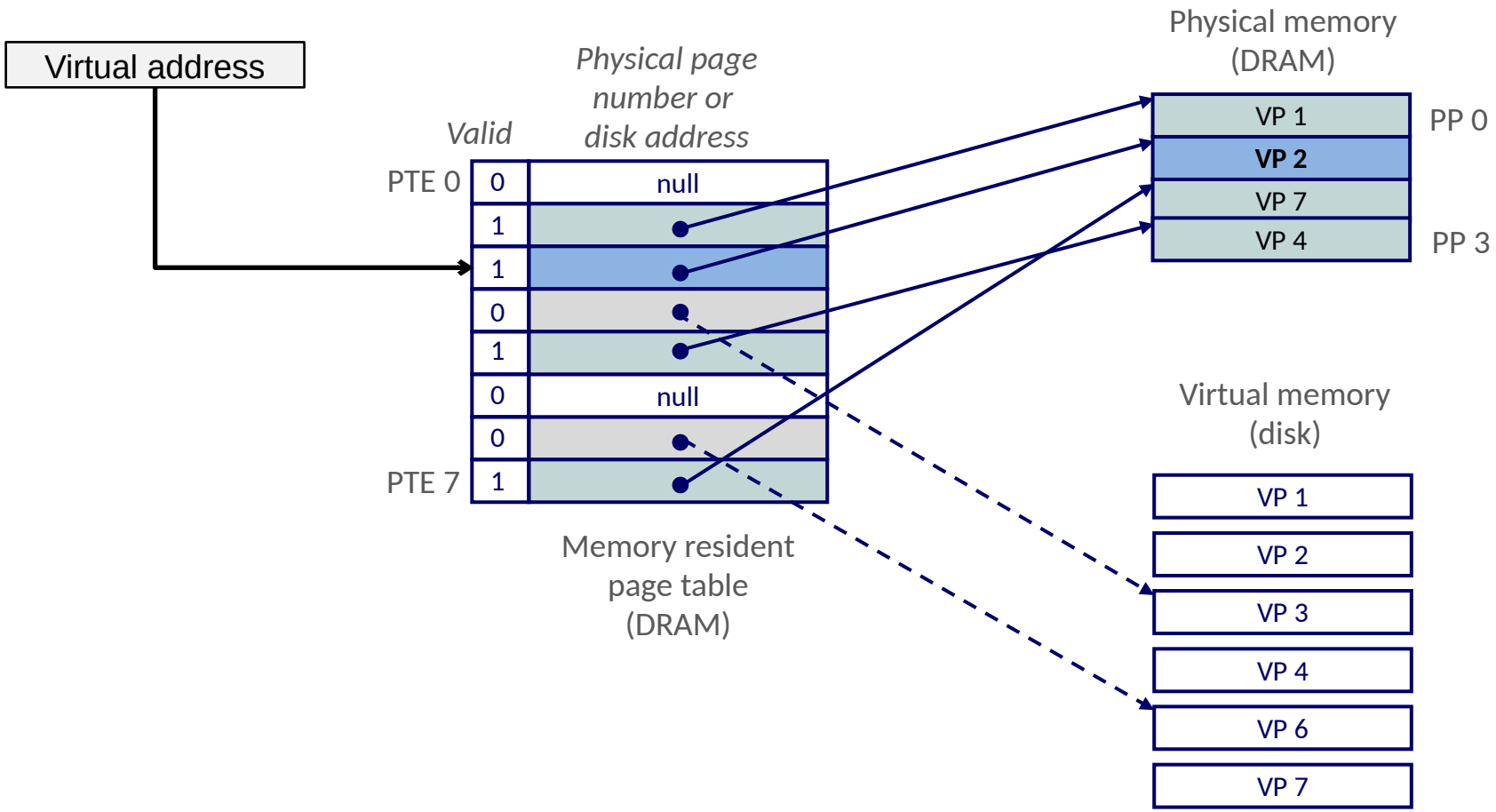
Page Hit

- *Page hit:* Access to a page that is in physical memory



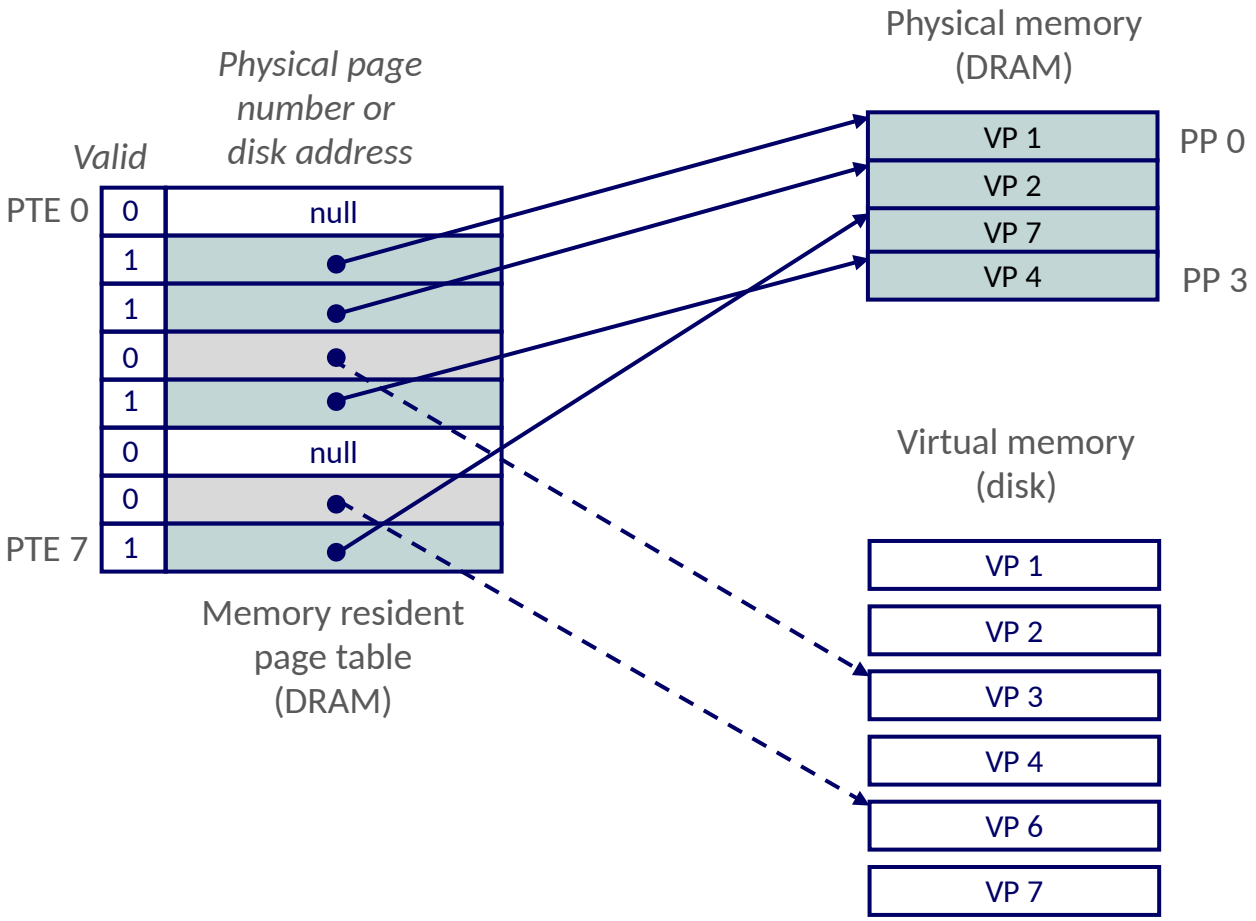
Page Hit

- *Page hit:* Access to a page that is in physical memory



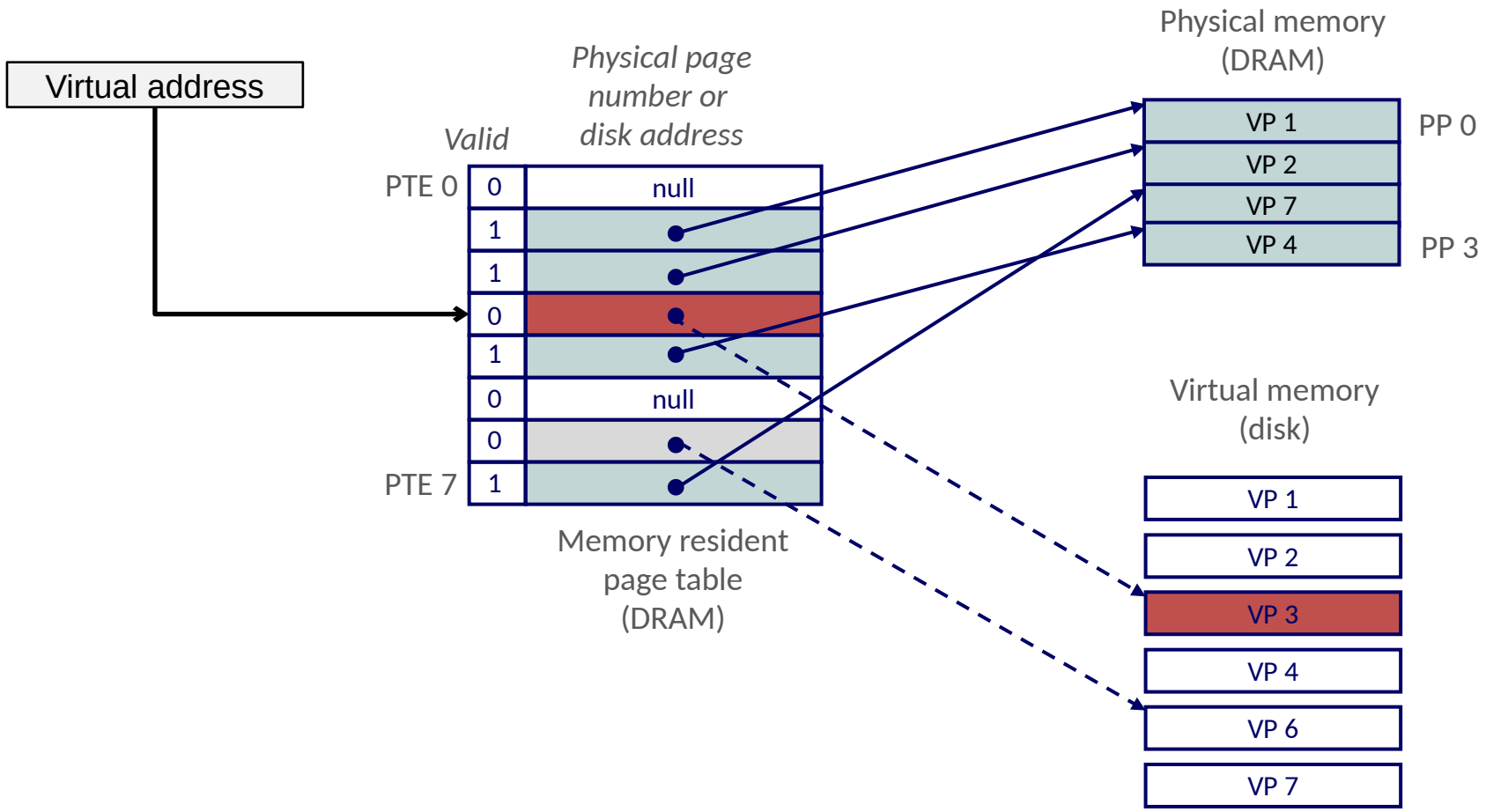
Page Fault

- *Page fault:* Access to a page that is *not* in physical memory



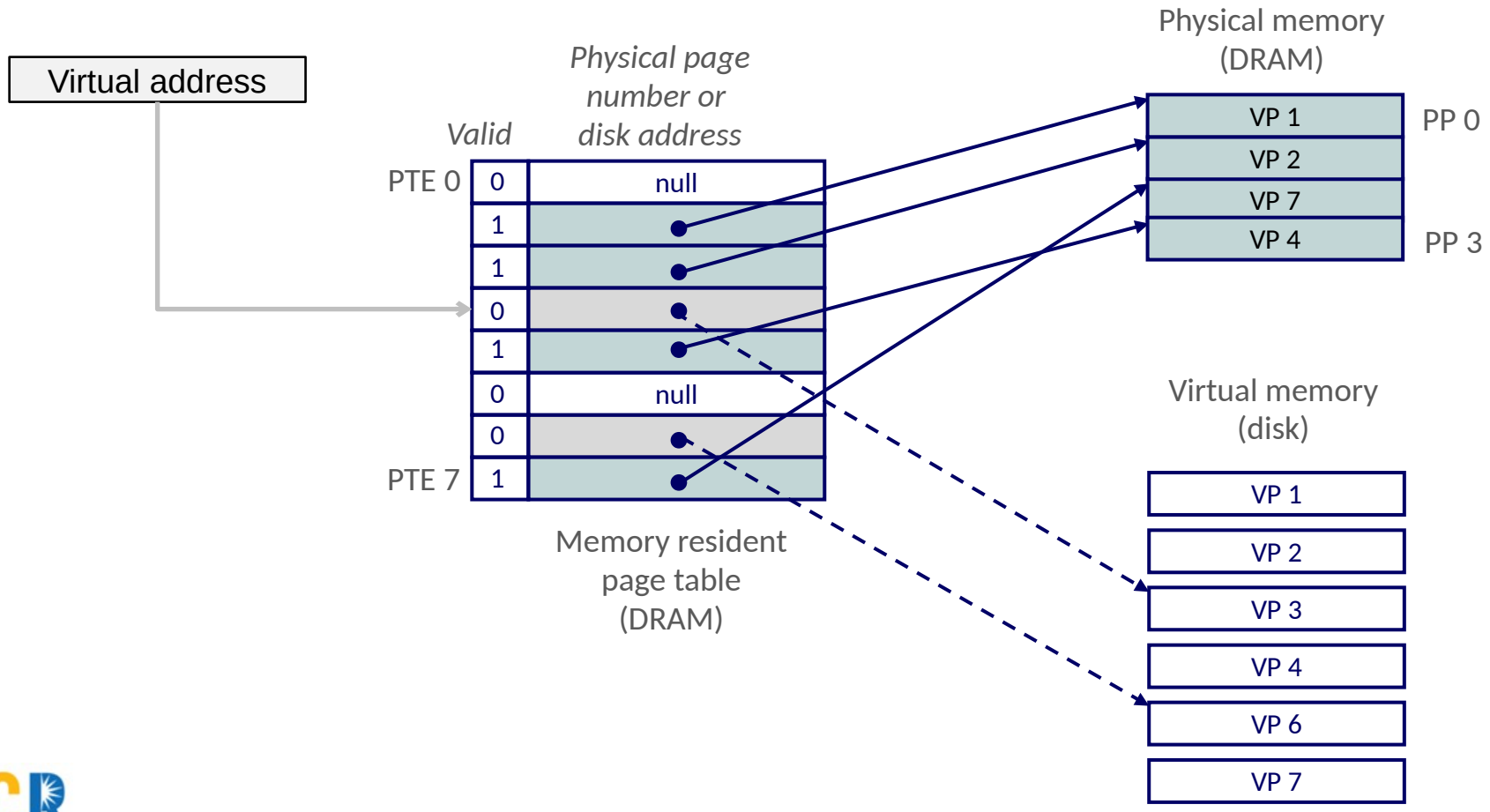
Page Fault

- *Page fault:* Access to a page that is *not* in physical memory



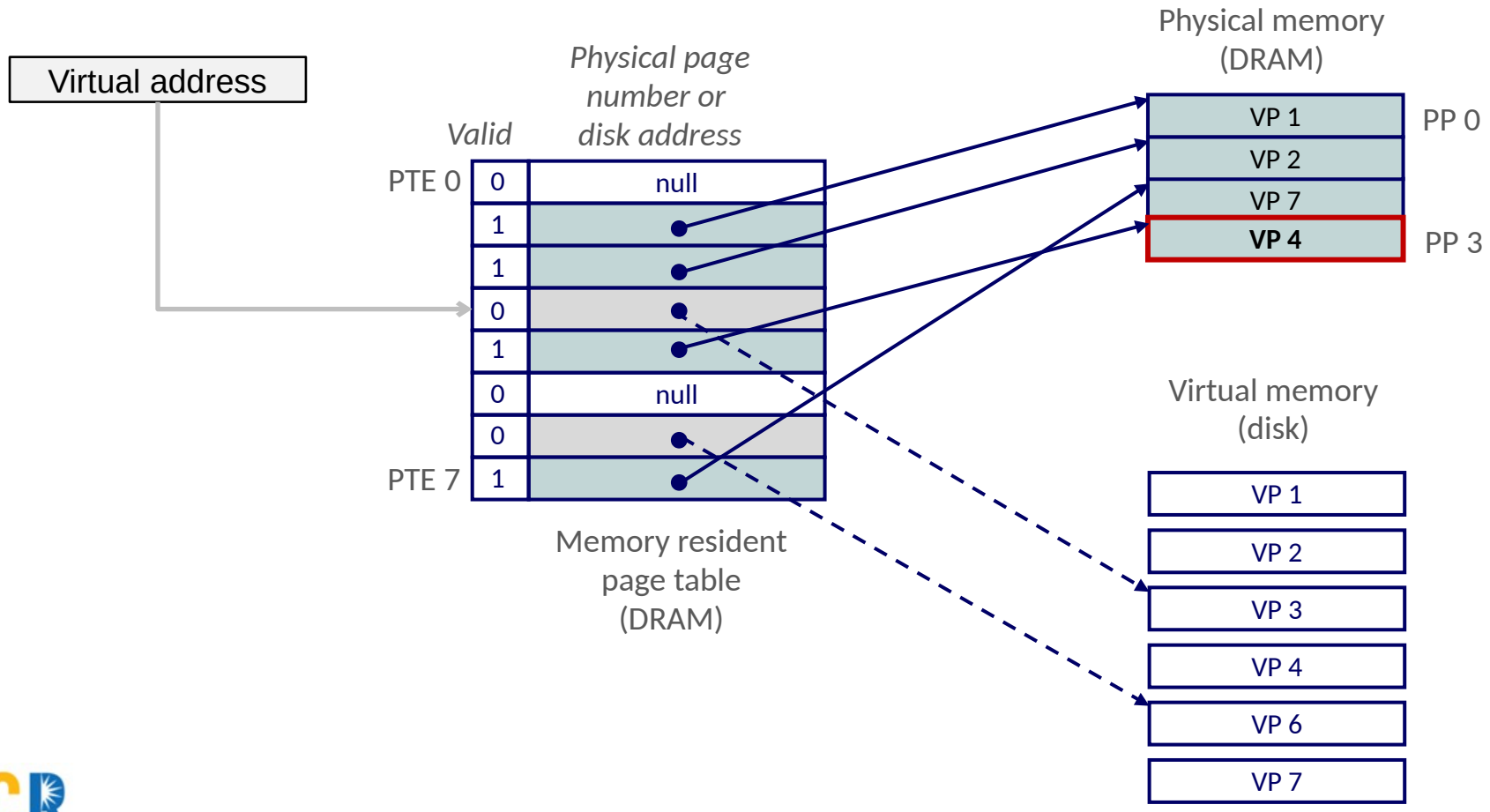
Handling Page Fault

- Page miss causes page fault (an exception)



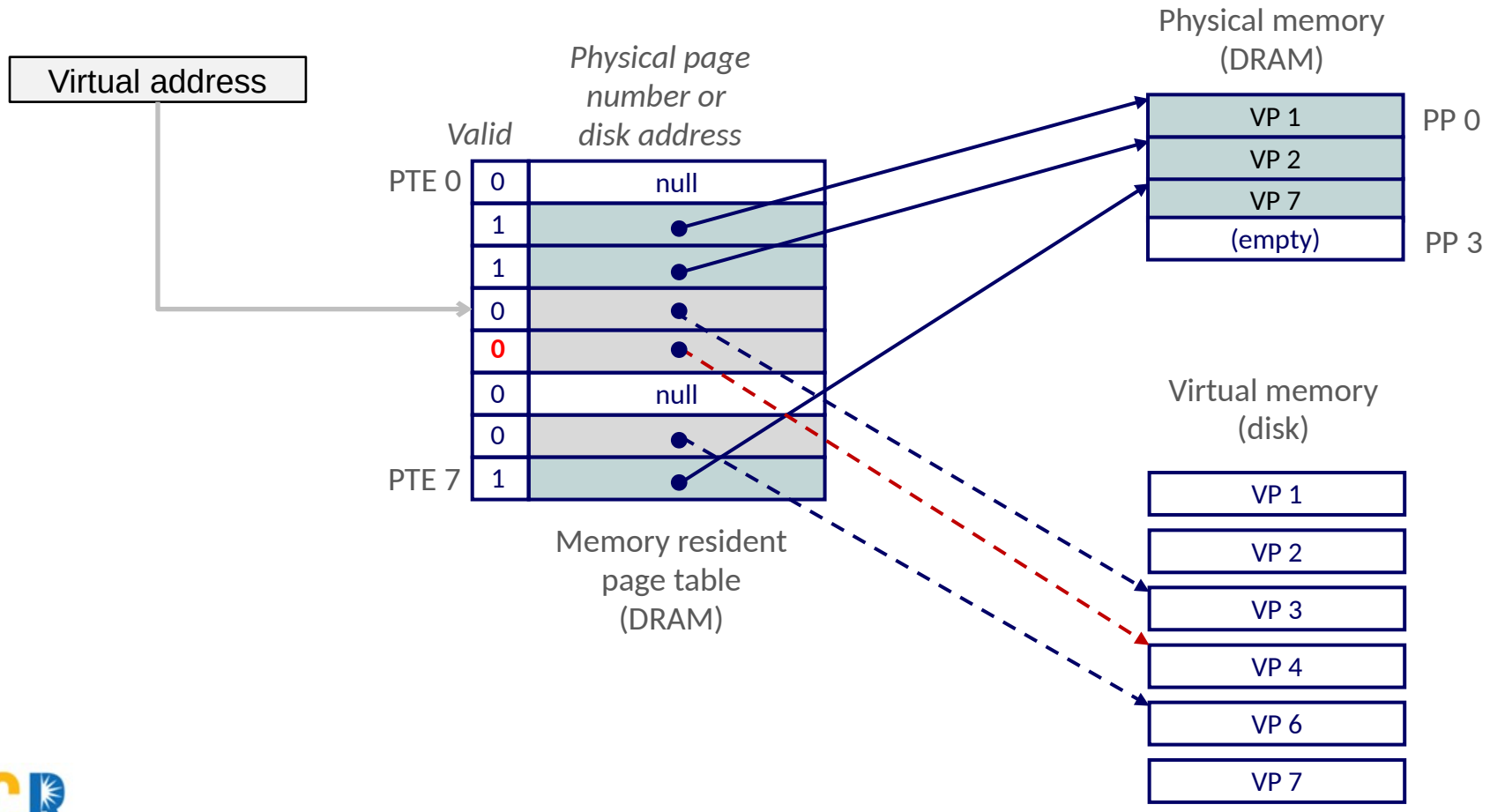
Handling Page Fault

- Page miss causes page fault (an exception)
- Page fault handler selects a victim to be evicted (here VP 4)



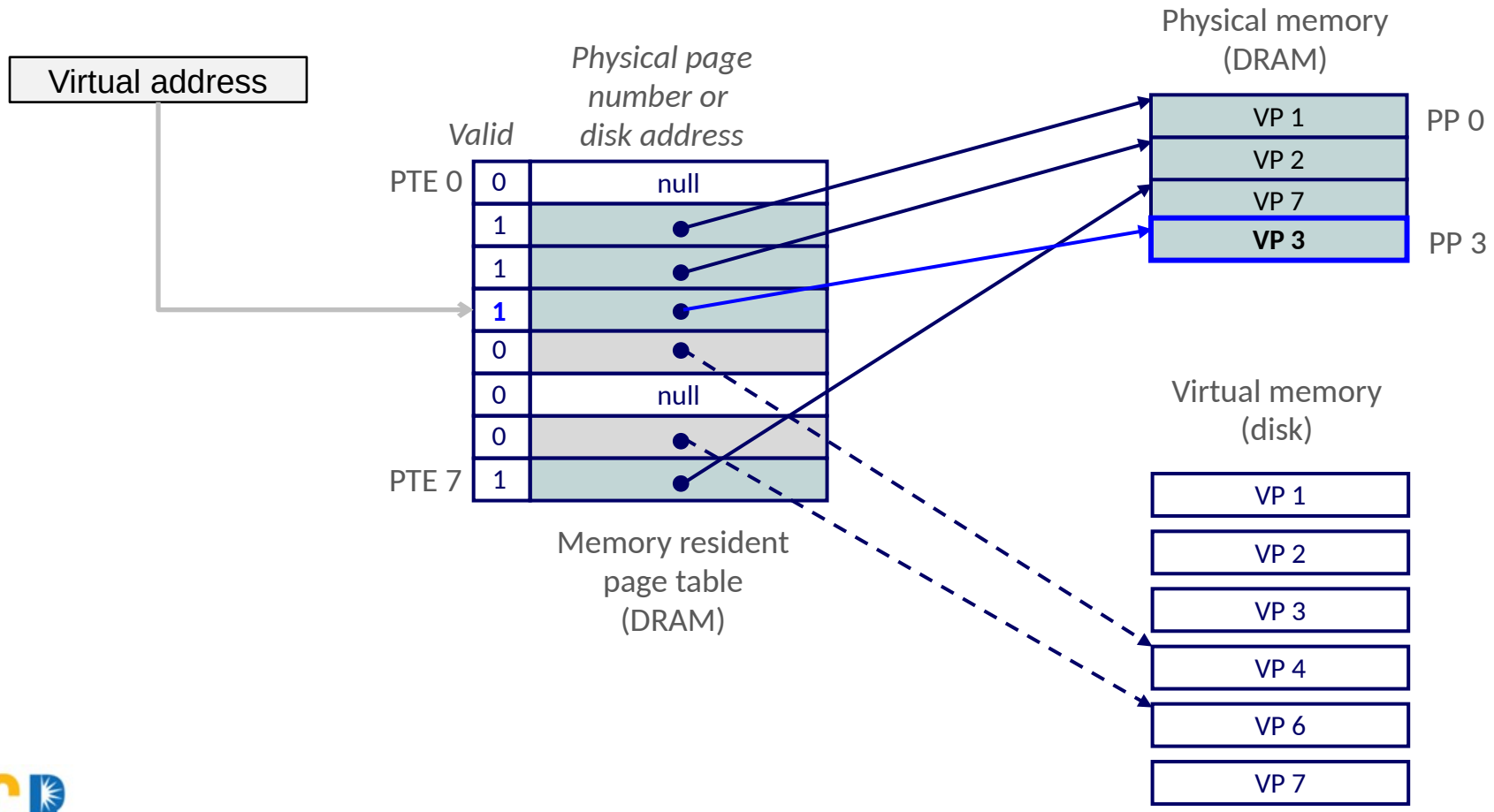
Handling Page Fault

- Page miss causes page fault (an exception)
- Page fault handler selects a victim to be evicted (here VP 4)



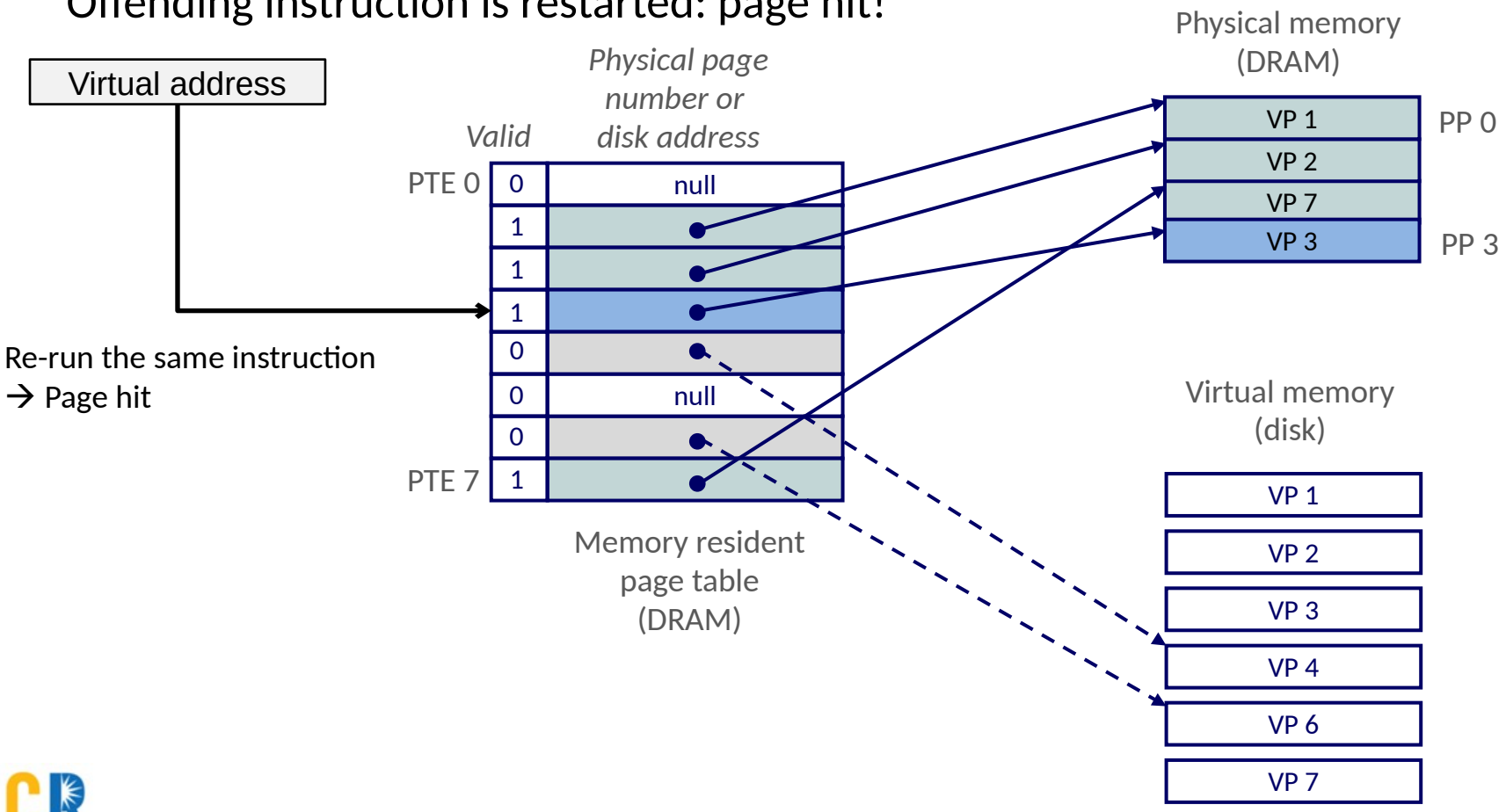
Handling Page Fault

- Page miss causes page fault (an exception)
- Page fault handler selects a victim to be evicted (here VP 4)



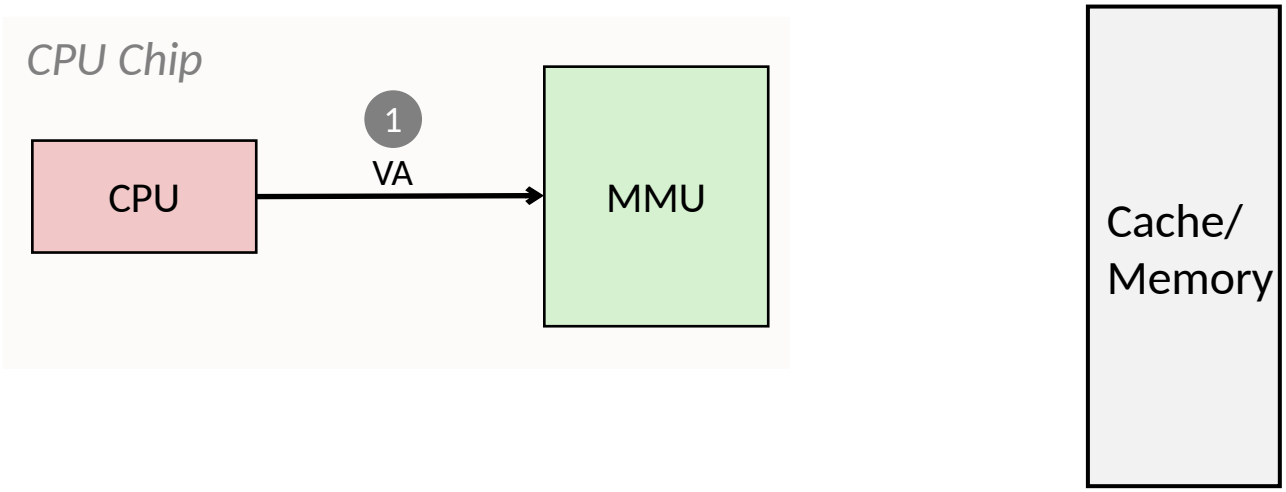
Handling Page Fault

- Page miss causes page fault (an exception)
- Page fault handler selects a victim to be evicted (here VP 4)
- Offending instruction is restarted: page hit!



Address Translation: Page Hit

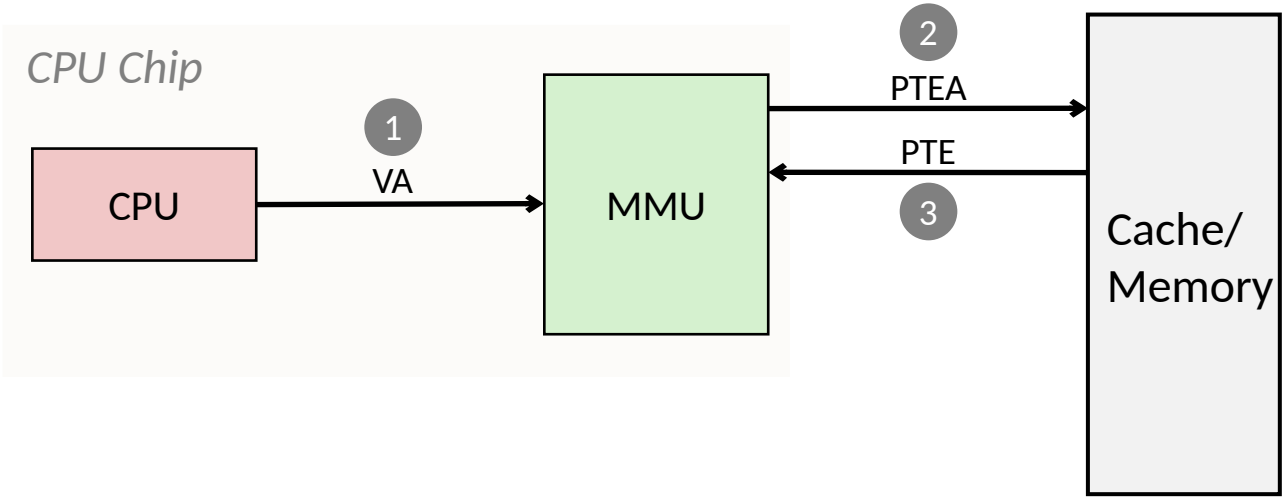
VA: virtual address, PA: physical address, PTE: page table entry, PTEA = PTE address



1) Processor sends virtual address to MMU

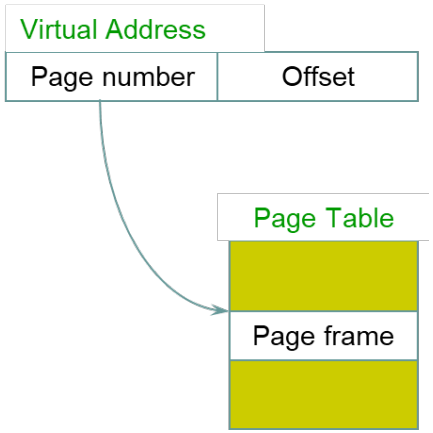
Address Translation: Page Hit

VA: virtual address, PA: physical address, PTE: page table entry, PTEA = PTE address



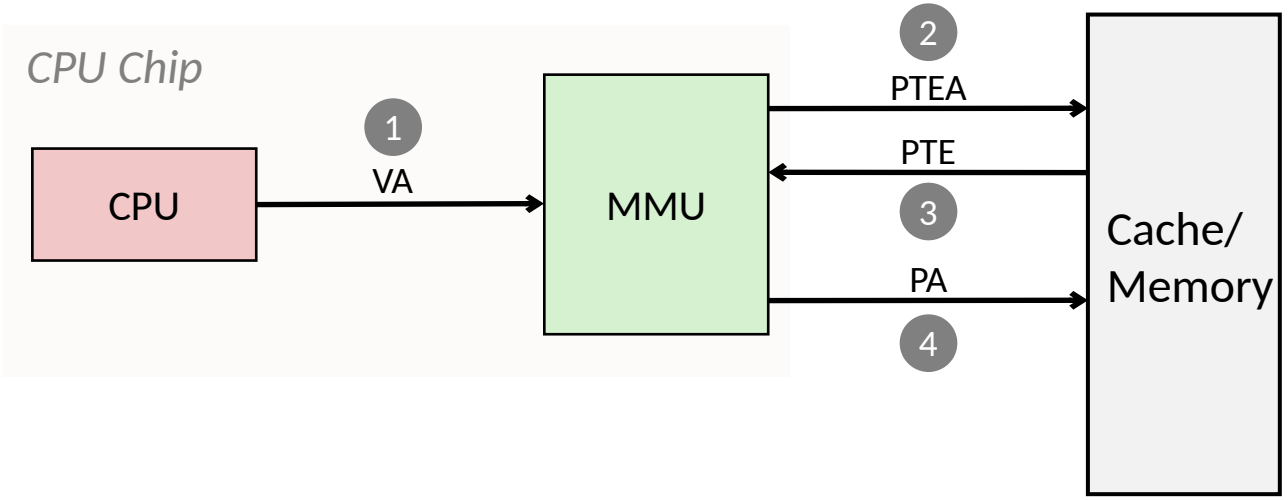
1) Processor sends virtual address to MMU

2-3) MMU fetches PTE from page table in memory



Address Translation: Page Hit

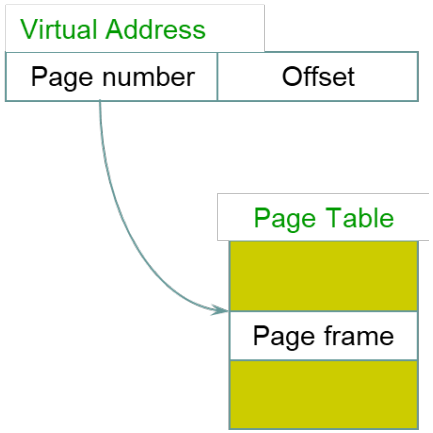
VA: virtual address, PA: physical address, PTE: page table entry, PTEA = PTE address



1) Processor sends virtual address to MMU

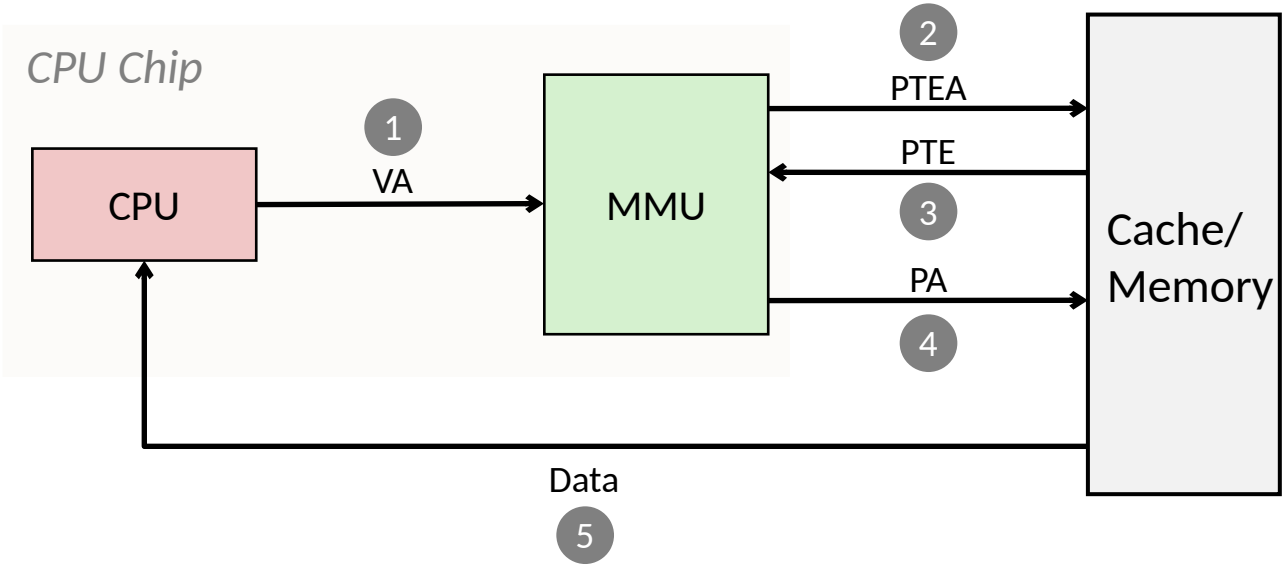
2-3) MMU fetches PTE from page table in memory

4) MMU sends physical address to cache/memory



Address Translation: Page Hit

VA: virtual address, PA: physical address, PTE: page table entry, PTEA = PTE address

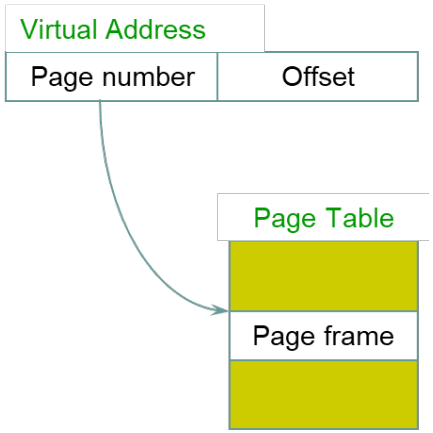


1) Processor sends virtual address to MMU

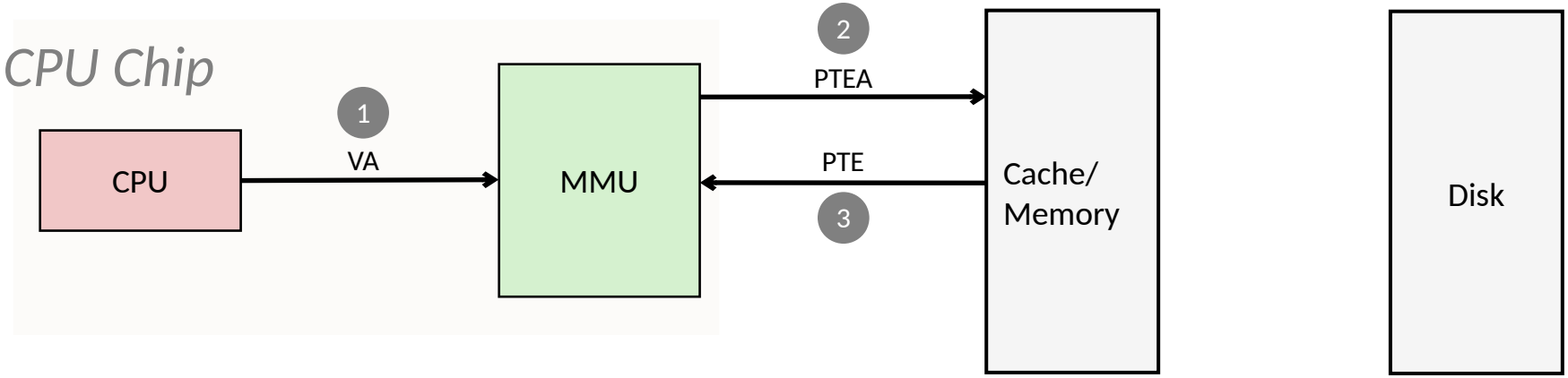
2-3) MMU fetches PTE from page table in memory

4) MMU sends physical address to cache/memory

5) Cache/memory sends data word to processor



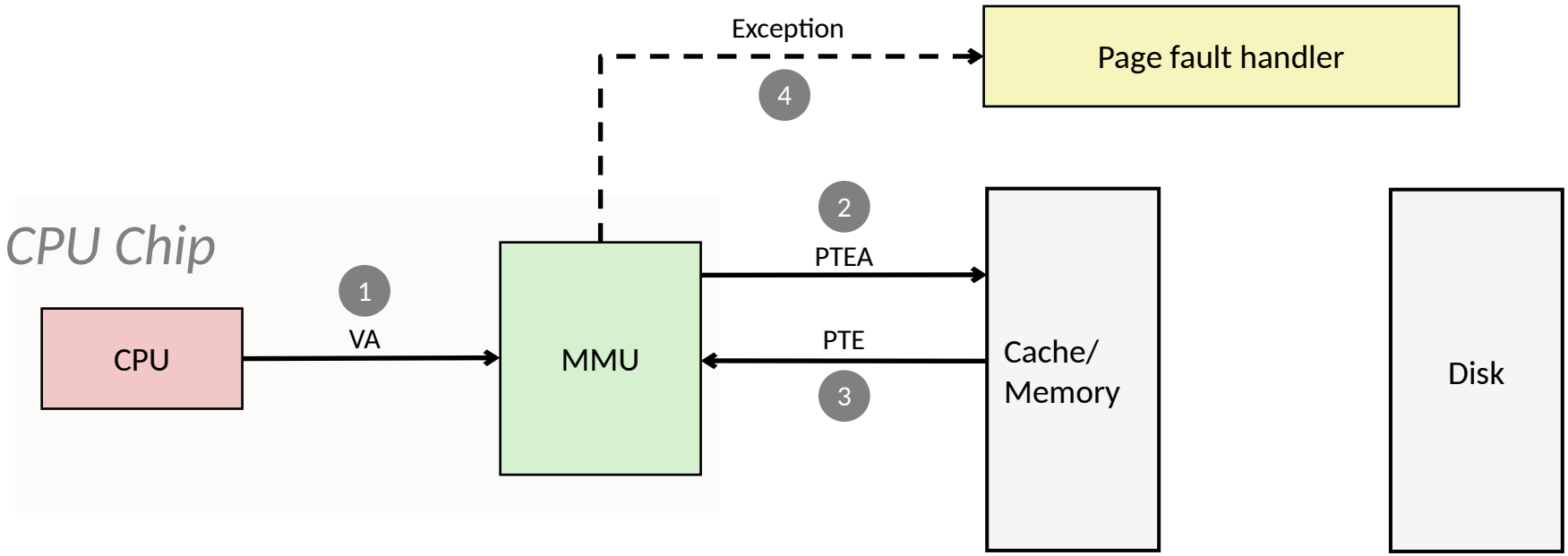
Address Translation: Page Fault



1) Processor sends virtual address to MMU

2-3) MMU fetches PTE from page table in memory

Address Translation: Page Fault

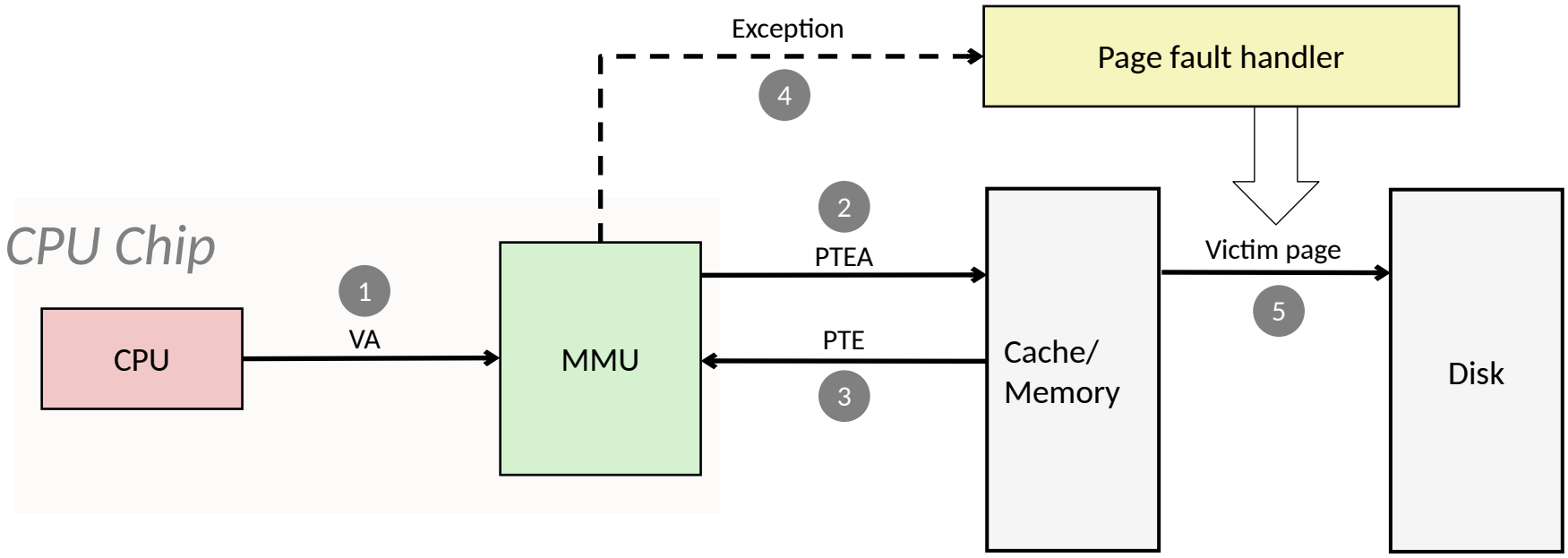


1) Processor sends virtual address to MMU

2-3) MMU fetches PTE from page table in memory

4) Valid bit is zero, so MMU triggers page fault exception

Address Translation: Page Fault



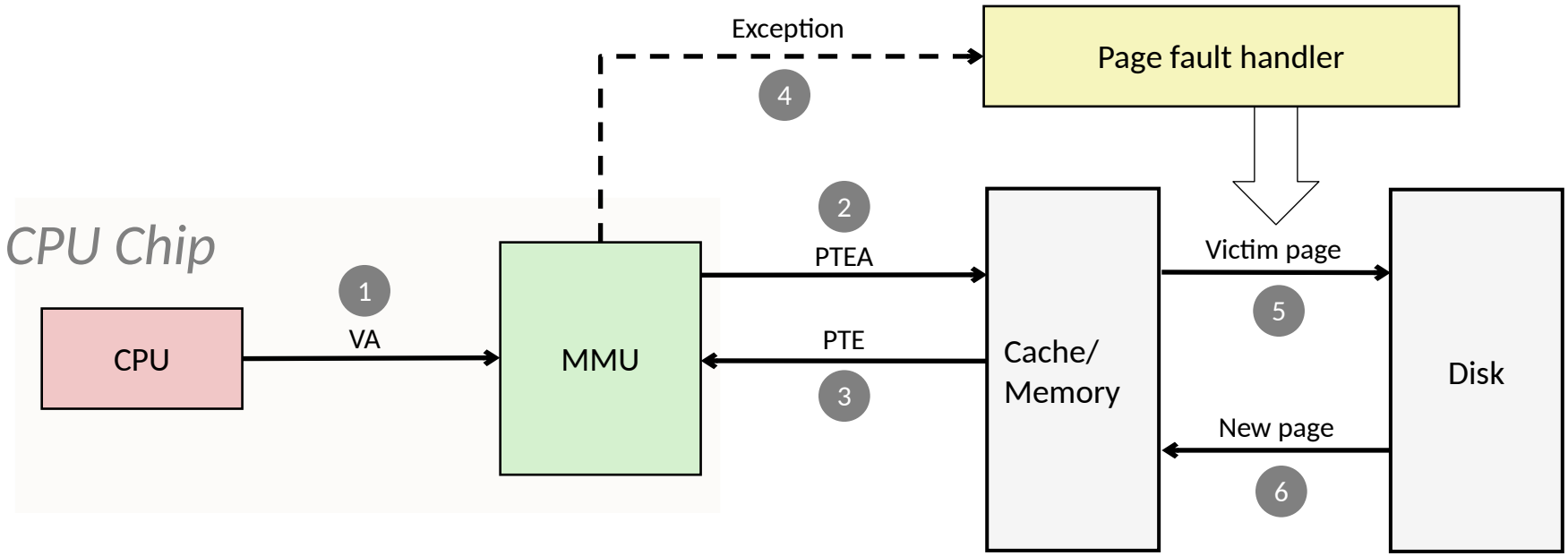
1) Processor sends virtual address to MMU

2-3) MMU fetches PTE from page table in memory

4) Valid bit is zero, so MMU triggers page fault exception

5) Handler identifies victim (and, if dirty, pages it out to disk)

Address Translation: Page Fault



1) Processor sends virtual address to MMU

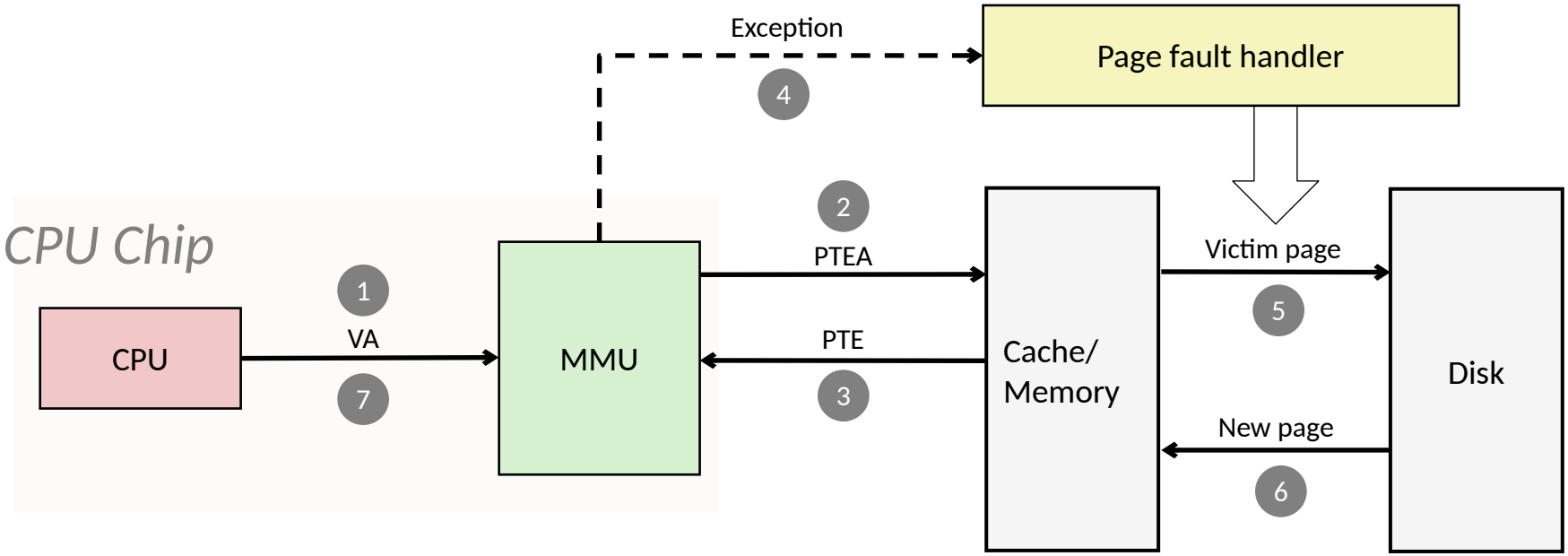
2-3) MMU fetches PTE from page table in memory

4) Valid bit is zero, so MMU triggers page fault exception

5) Handler identifies victim (and, if dirty, pages it out to disk)

6) Handler pages in new page and updates PTE in memory

Address Translation: Page Fault



1) Processor sends virtual address to MMU

2-3) MMU fetches PTE from page table in memory

4) Valid bit is zero, so MMU triggers page fault exception

5) Handler identifies victim (and, if dirty, pages it out to disk)

6) Handler pages in new page and updates PTE in memory

7) Handler returns to original process, restarting faulting instruction

Overhead due to a page fault

- PTE informs the page is on disk: 1 cycle
- CPU generates a page fault exception: 100 cycles
- OS page fault handler called
  - OS chooses a page to evict from DRAM and write to disk: 10k cycles
  - Write dirty page back to disk first: 40m cycles
  - OS then reads the page from disk and put it in DRAM: 40m cycles
  - OS changes the page table to map the new page: 1k cycles
- OS resumes the instruction that caused the page faul: 10k cycles
- Page faults are the **slowest** thing (except for human interactions)
- Interestingly, there are systems do not page:
  - iOS: kills the program if using too much memory

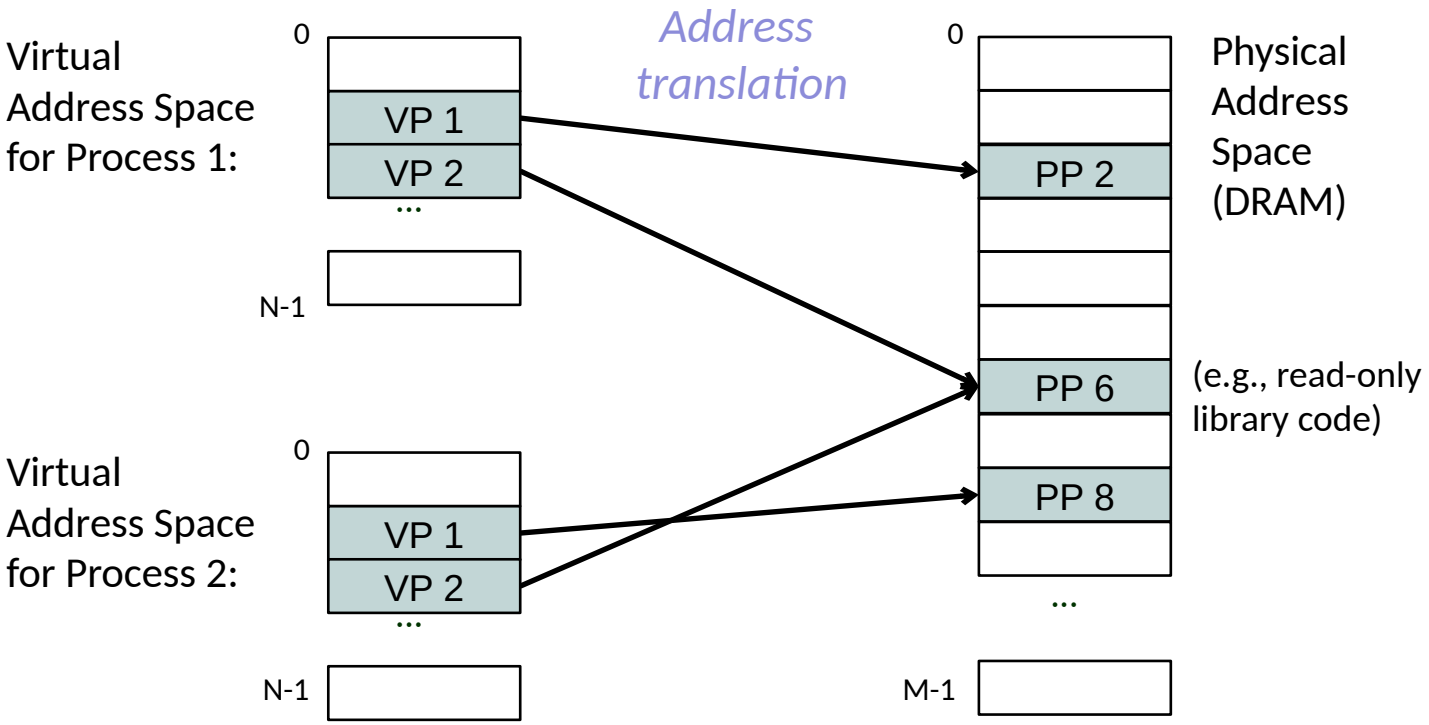
Discussion Questions

Why is the same instruction retried after a page fault is handled?

A. To ensure fairness across processes
B. Because the MMU needs two cycles to confirm a hit
C. Because the faulting memory address becomes valid once the page is loaded
D. Because the OS must flush the cache before moving on

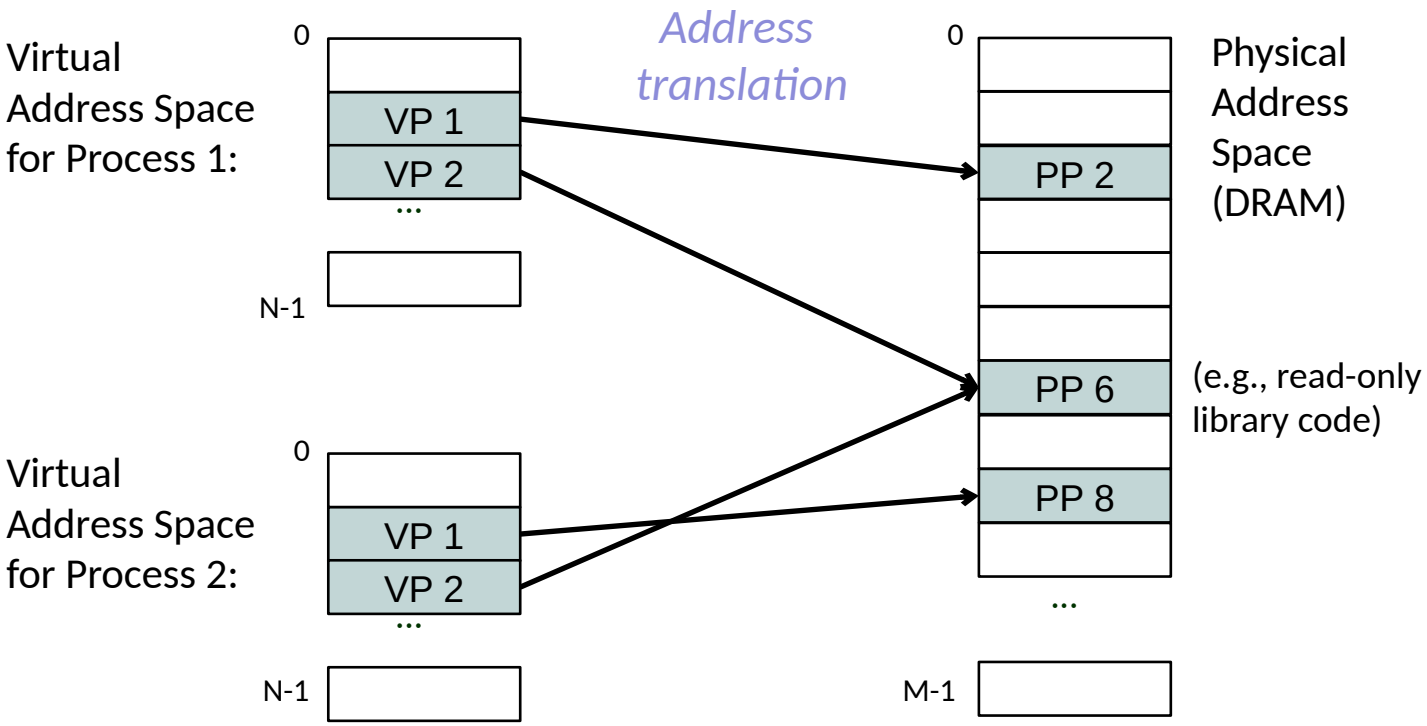
VM as a Tool for Mem Management

- Each process has its own virtual address space
  - It can view memory as a simple linear array
  - Mapping function scatters addresses through physical memory
    - Well chosen mappings simplify memory allocation and management



VM as a Tool for Mem Management

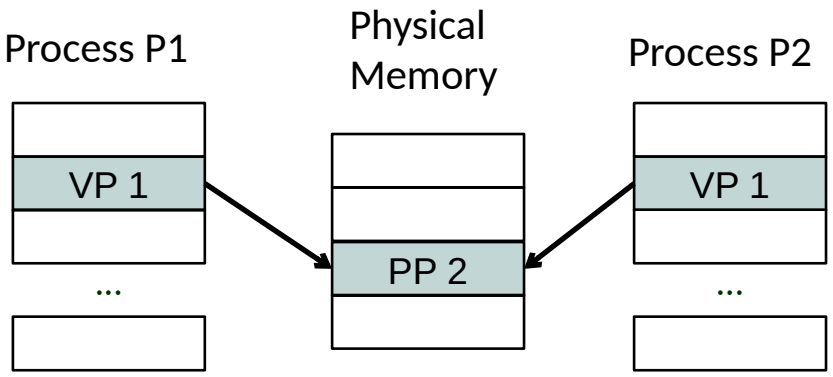
- Memory allocation
  - Each virtual page can be mapped to any physical page
  - A virtual page can be stored in different physical pages at different times
- Sharing code and data among processes
  - Map virtual pages to the same physical page (here: PP 6)



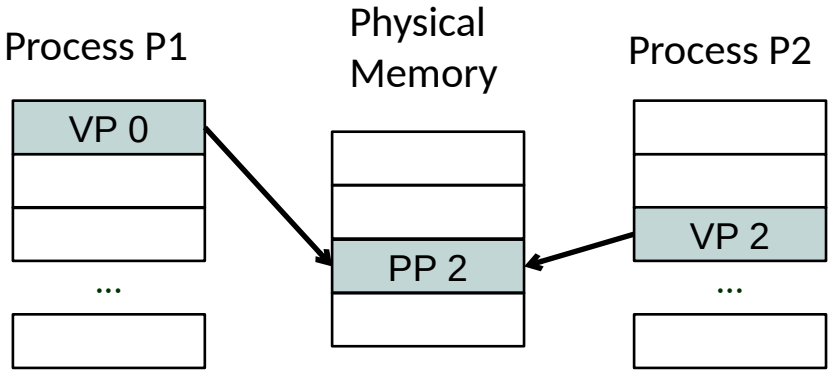
Sharing

- Can map shared physical memory at same or different virtual addresses in each process' address space

  - Same VA:
    - Both P1 and P2 maps the 1st virtual page (VP1) to the shared physical page (PP2)

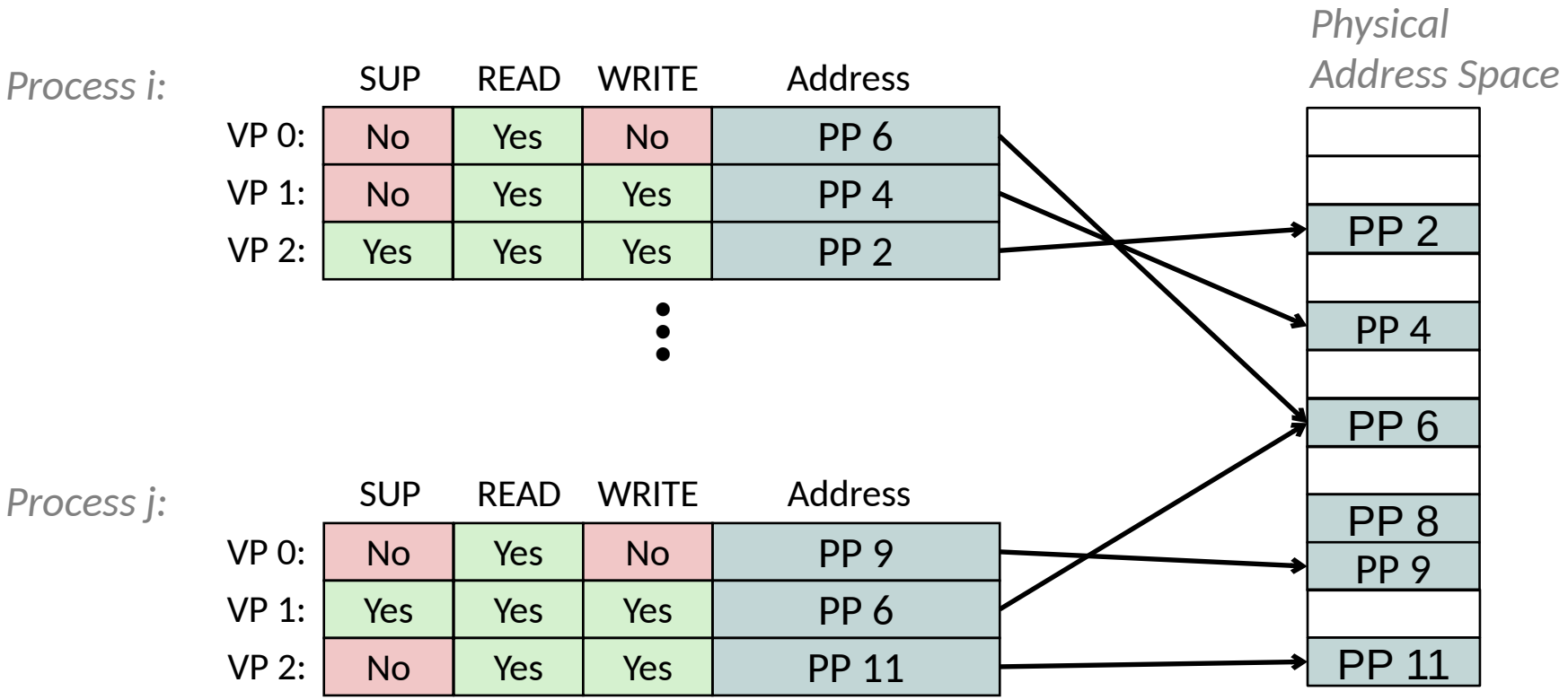


  - Different VA:
    - P1 maps VP0 to PP2, but P2 maps VP2 to PP2



VM as a Tool for Mem Protection

- Extend PTEs with permission bits
- Page fault handler checks these before remapping
  - If violated, send process SIGSEGV (segmentation fault)



* SUP = supervisor mode (kernel mode)

Discussion Questions

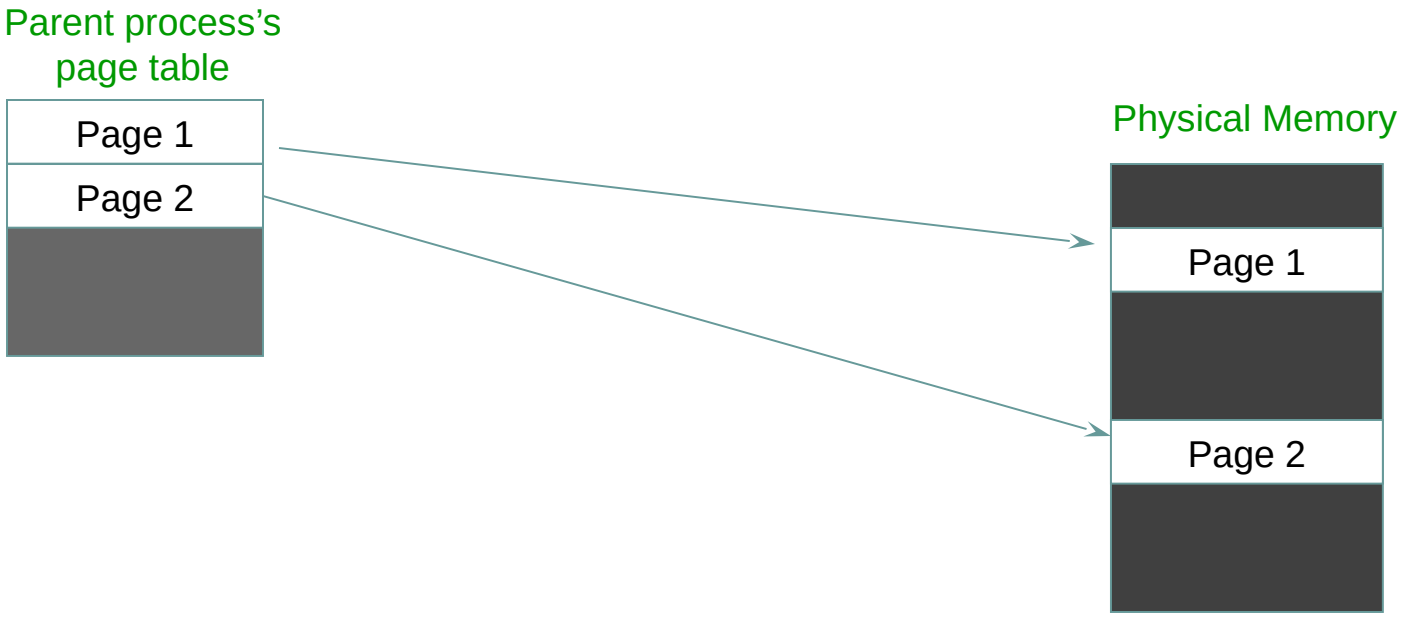
Which of the following best explains why a process may experience a second page fault immediately after the first one was resolved by loading the page from disk?

A. The page was incorrectly loaded into physical memory.
B. The TLB was not flushed and still contains an old, invalid entry.
C. The instruction that caused the first page fault is retried and fails again due to access protection.
D. The disk block was not cached, forcing a second fetch.

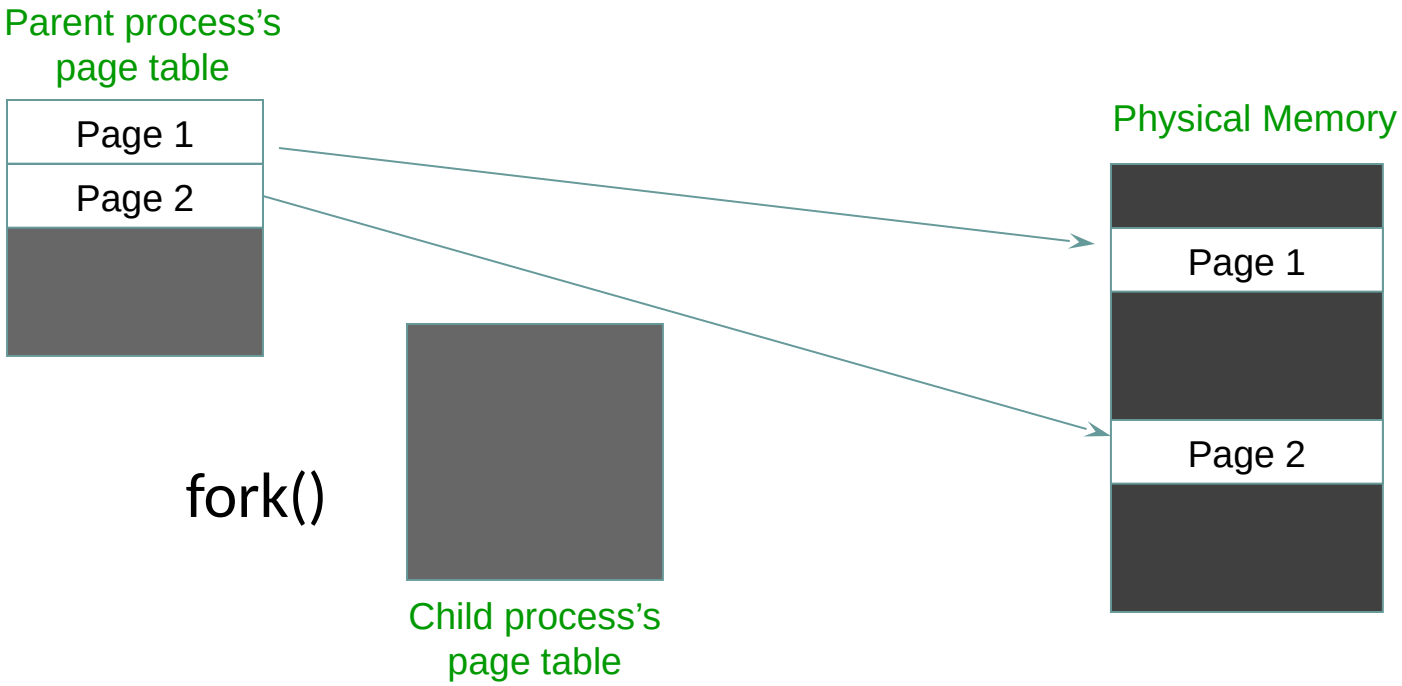
Copy on Write

- OSes spend a lot of time copying data
  - System call arguments between user/kernel space
  - Entire address spaces to implement fork()
- Use Copy on Write (CoW) to defer large copies as long as possible, hoping to avoid them altogether
  - Instead of copying pages, create shared mappings of parent pages in child virtual address space
  - Shared pages are protected as read-only in parent and child
    - Reads happen as usual
    - Writes generate a protection fault, trap to OS, copy page, change page mapping in client page table, restart write instruction
  - How does this help fork()?

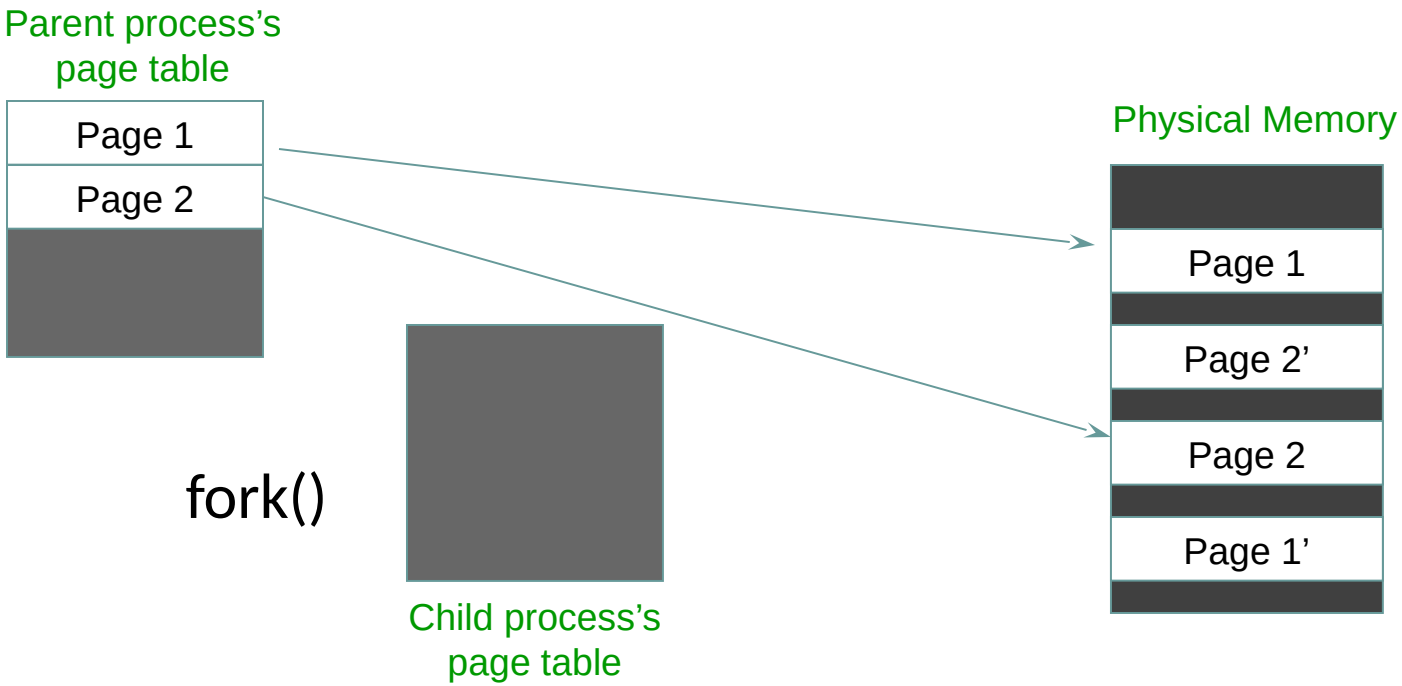
fork() without Copy on Write



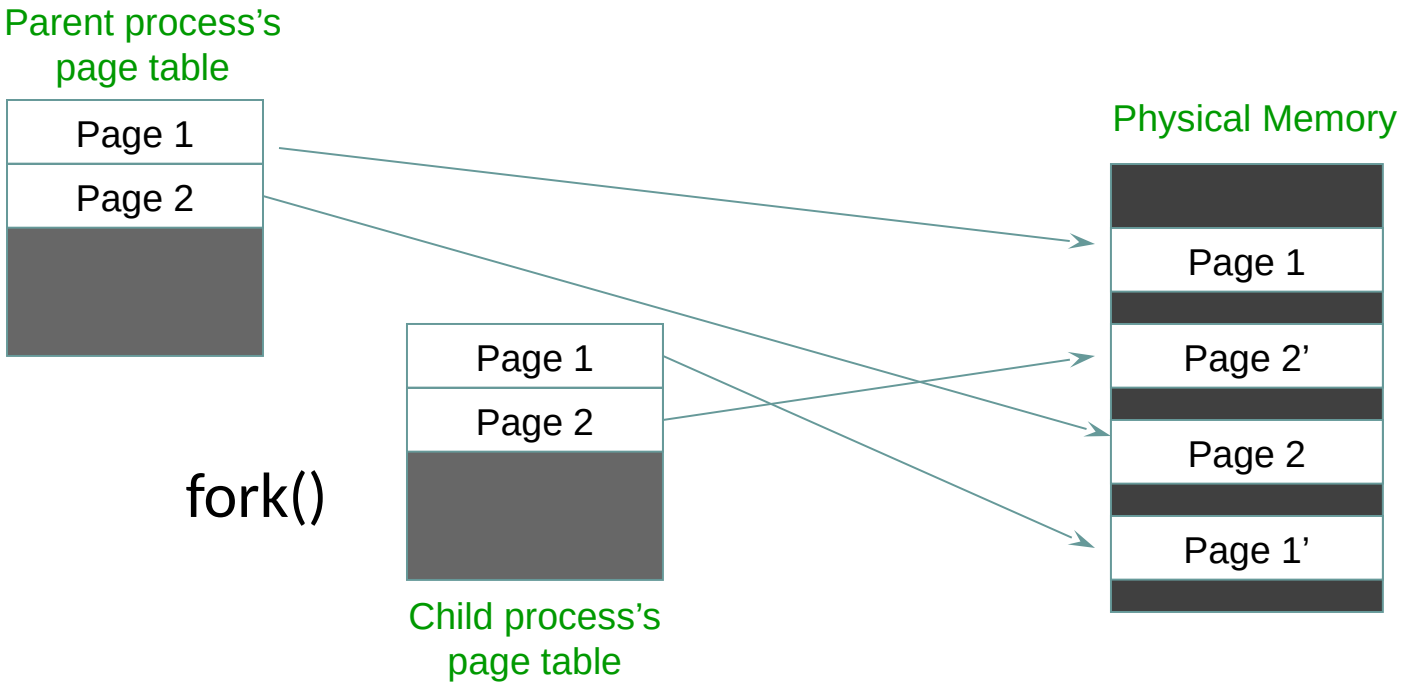
fork() without Copy on Write



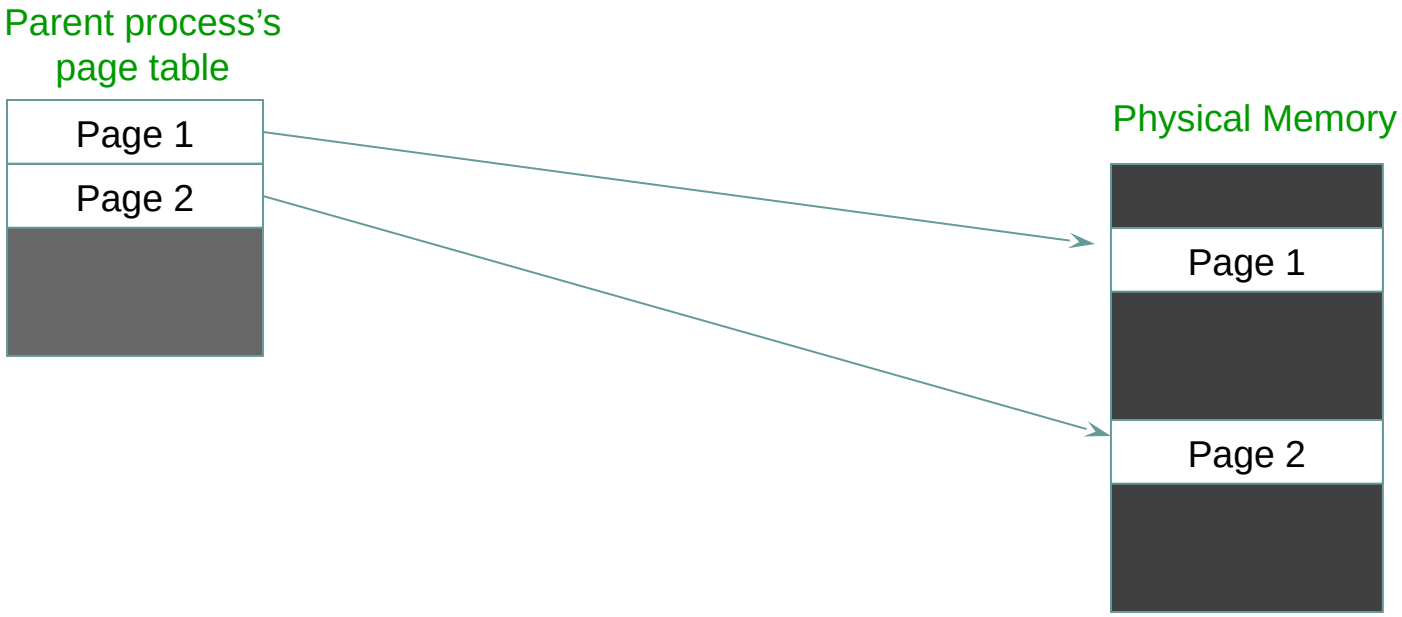
fork() without Copy on Write



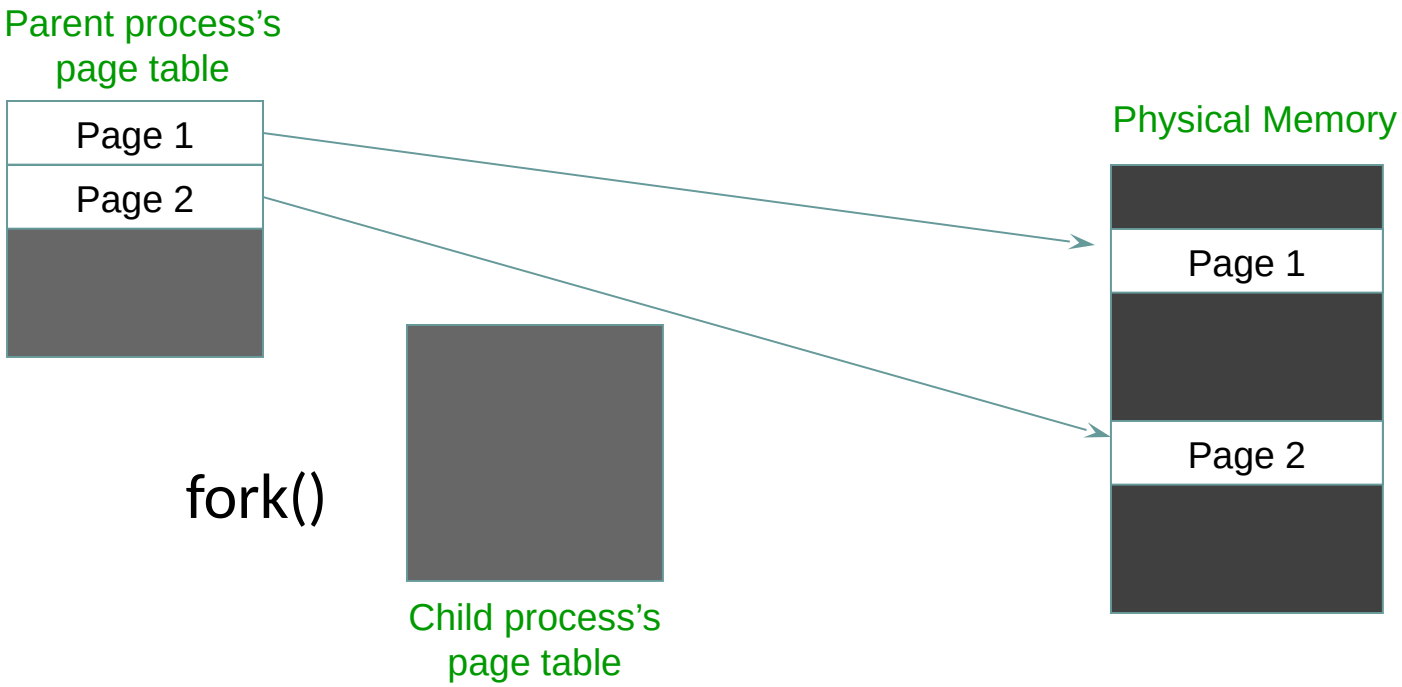
fork() without Copy on Write



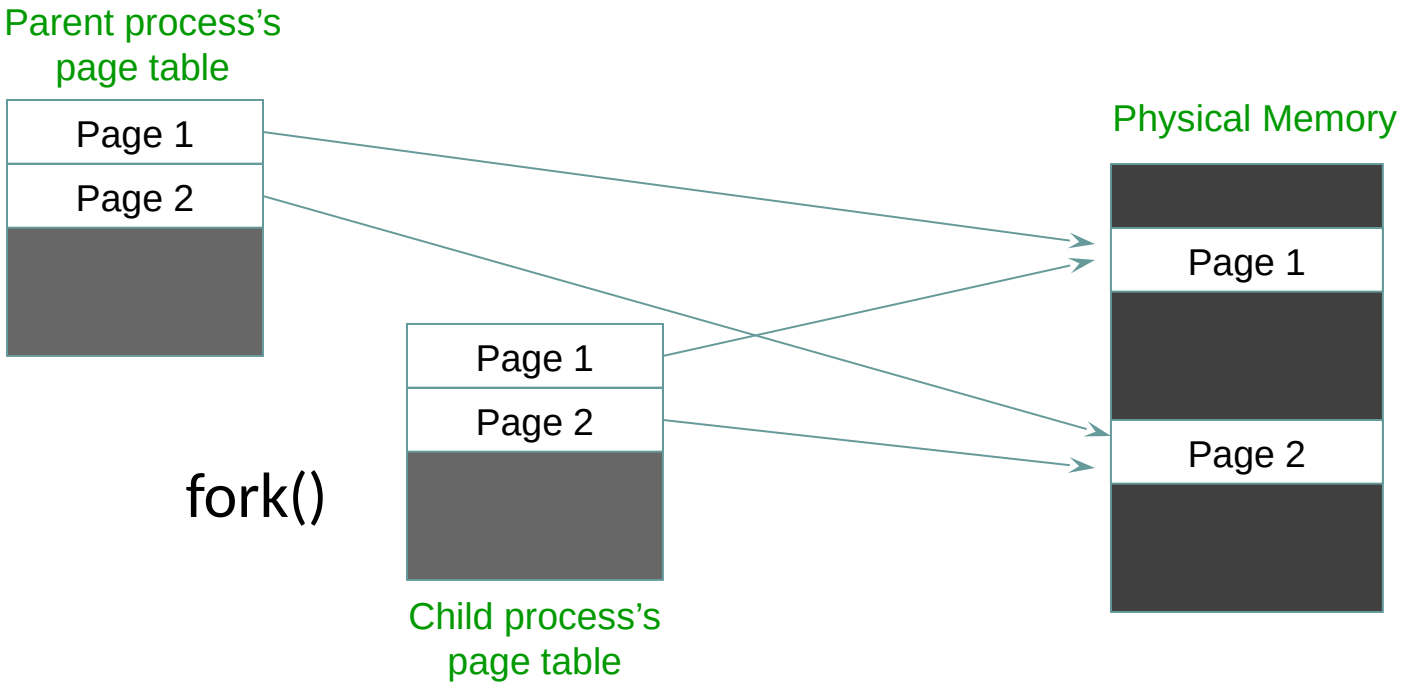
fork() with Copy on Write



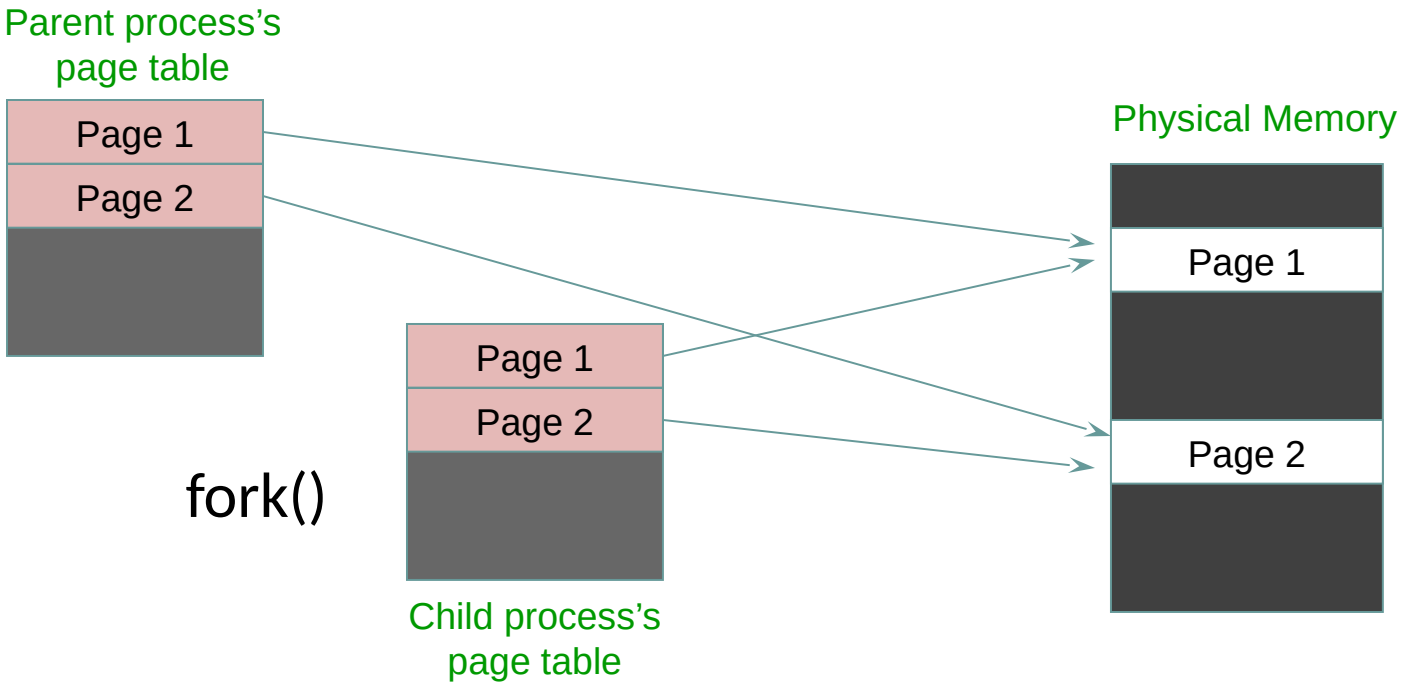
fork() with Copy on Write



fork() with Copy on Write

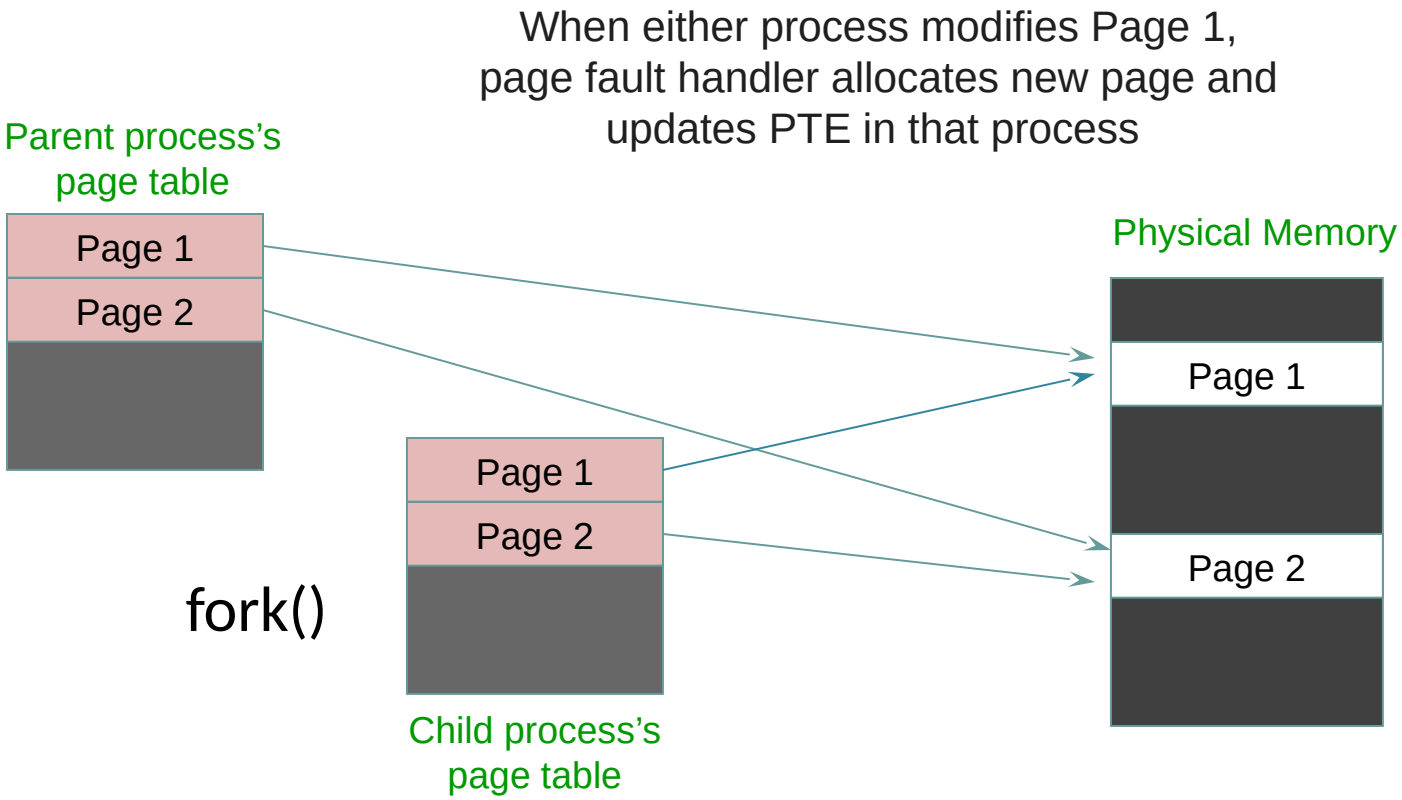


fork() with Copy on Write

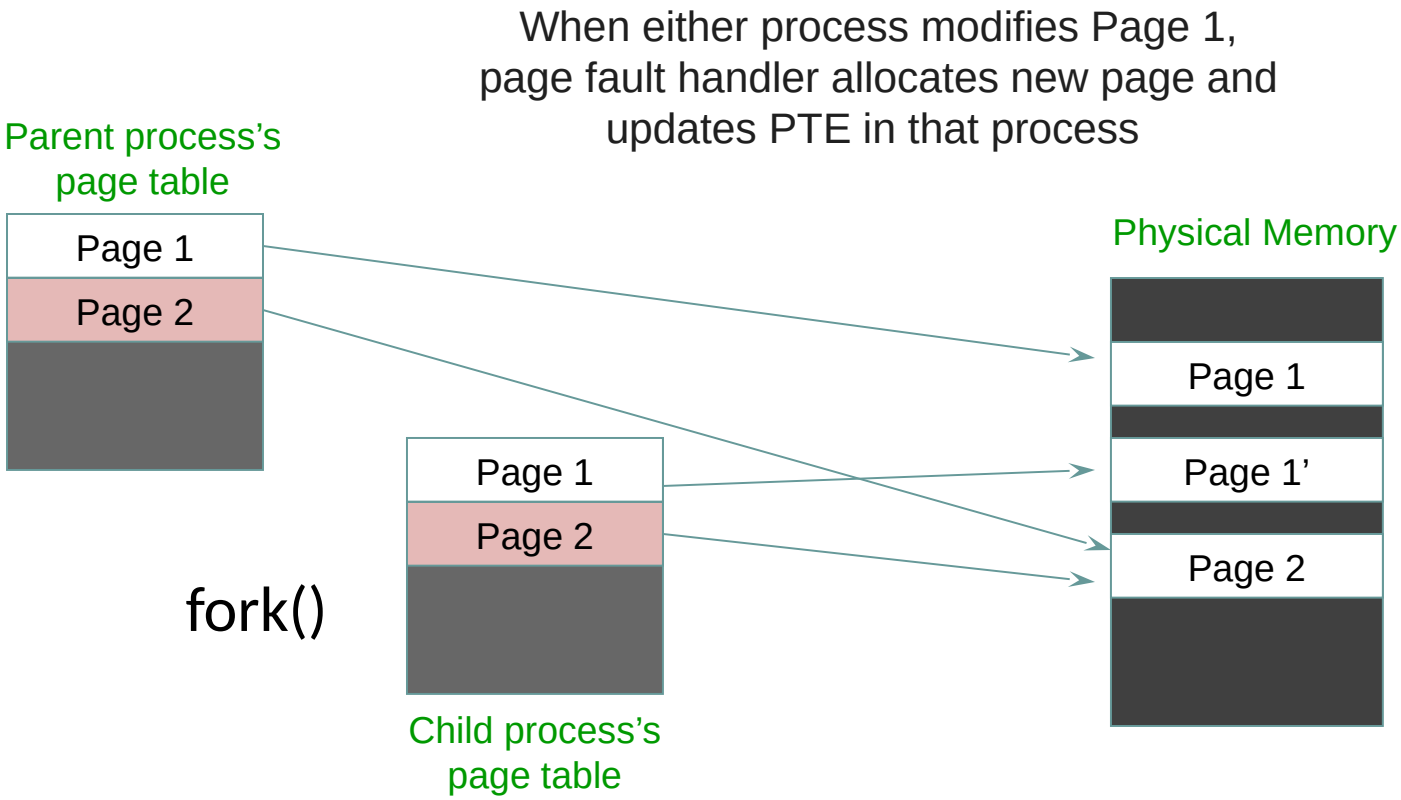


Protection bits set to prevent either process from writing to any page

fork() with Copy on Write



fork() with Copy on Write



Elephant(s) in the room

Elephant(s) in the room

- **Problem 1: Translation is slow!**
  - Many memory accesses for each memory access
  - L1 cache is not effective



"I suppose I'll be the one
to mention the elephant in the room."

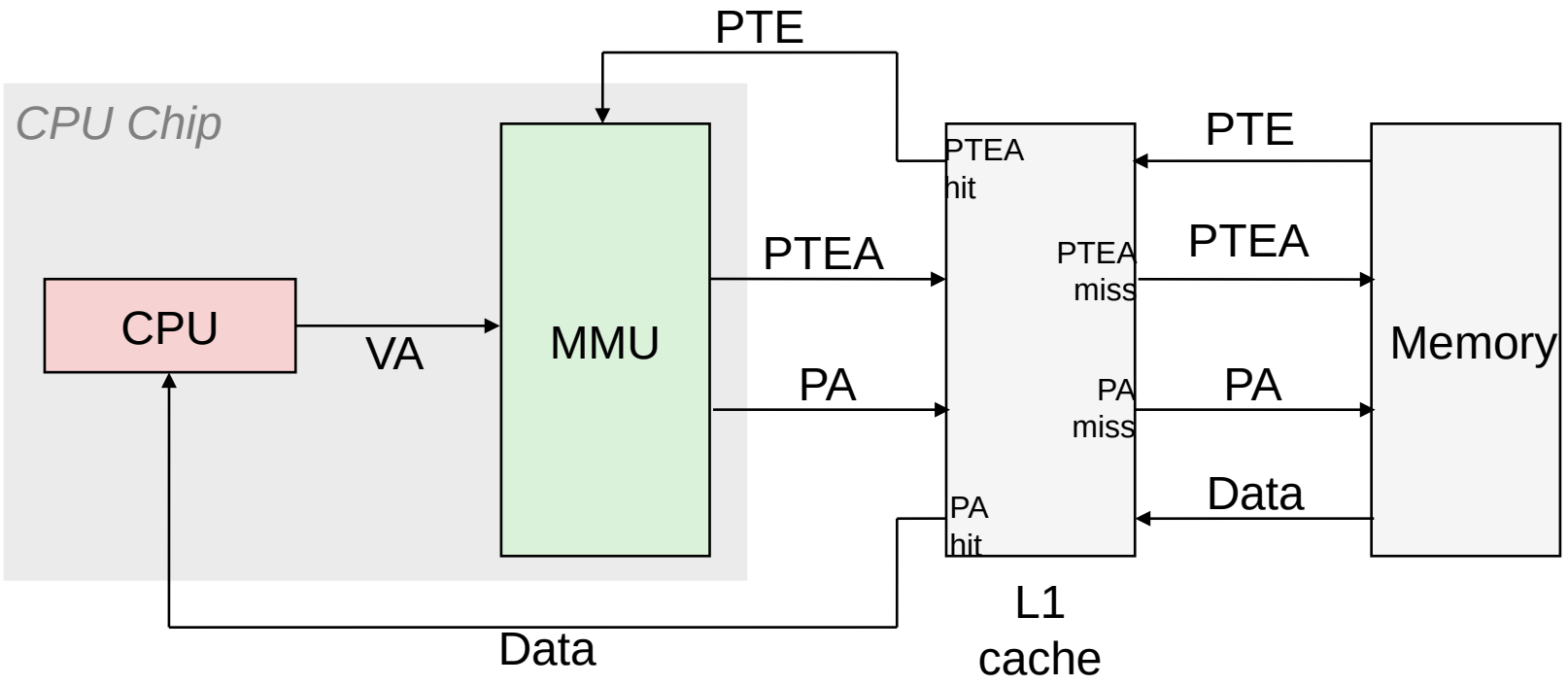
Elephant(s) in the room

- **Problem 1: Translation is slow!**
  - Many memory accesses for each memory access
  - L1 cache is not effective
- **Problem 2: Page table can be gigantic!**
  - We need one for each process



"Unfortunately, there's another elephant in the room."

Integrating VM and Cache



VA: virtual address, PA: physical address, PTE: page table entry, PTEA = PTE address

Speeding up Translation with a TLB

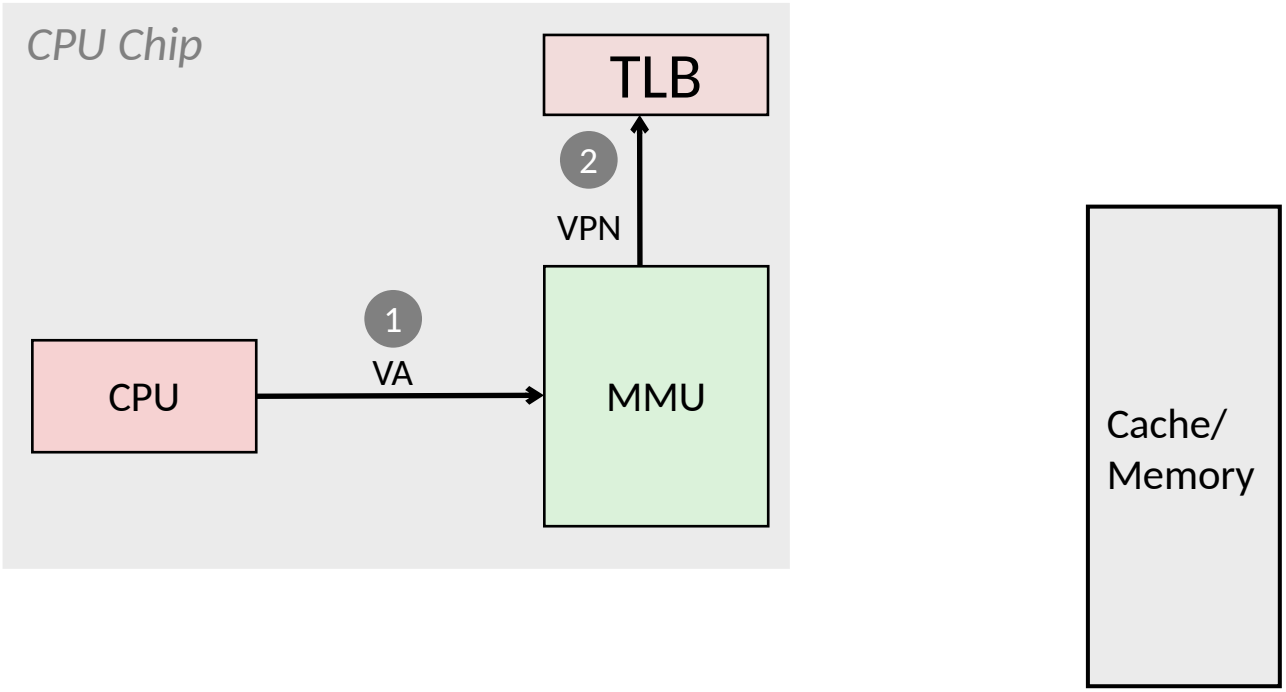
- If page table entries (PTEs) are cached in L1 like any other data
  - PTEs may be evicted by other data references
  - PTE hit still requires a small L1 access delay

Speeding up Translation with a TLB

- If page table entries (PTEs) are cached in L1 like any other data
  - PTEs may be evicted by other data references
  - PTE hit still requires a small L1 access delay

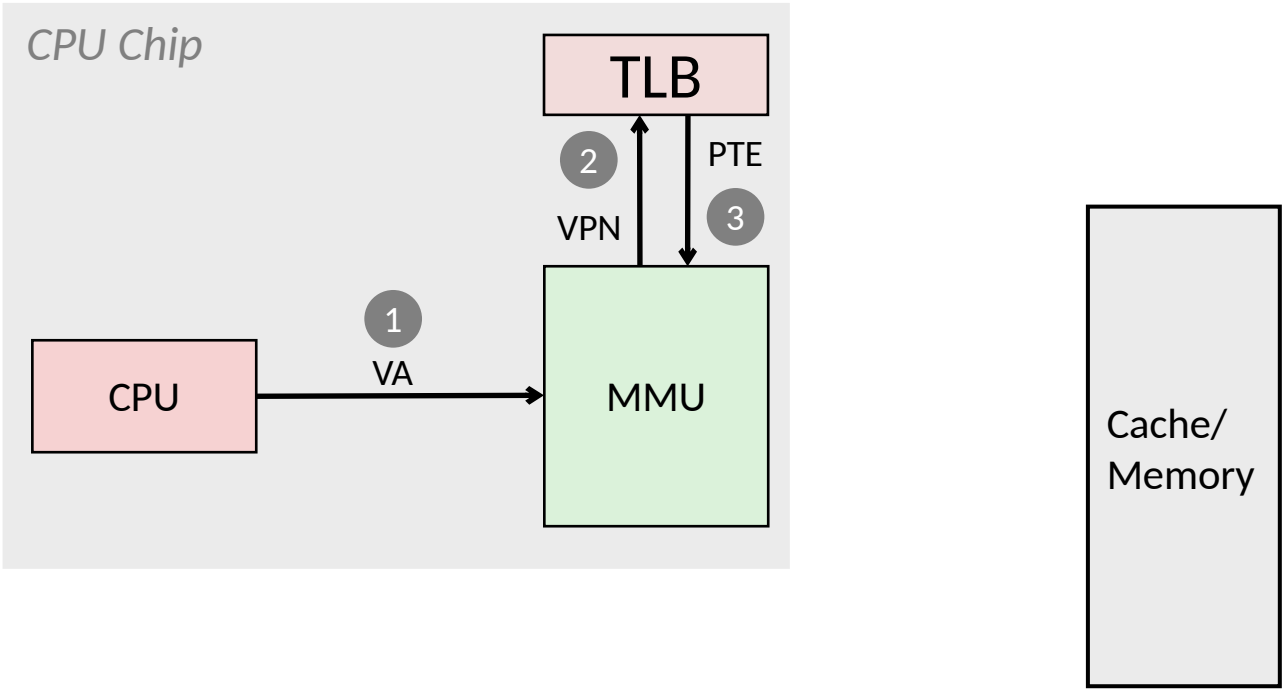
- Solution: *Translation Lookaside Buffer* (TLB)
  - Small hardware cache in MMU
  - Maps virtual page numbers to physical page numbers
  - Contains complete page table entries for small number of pages

TLB Hit



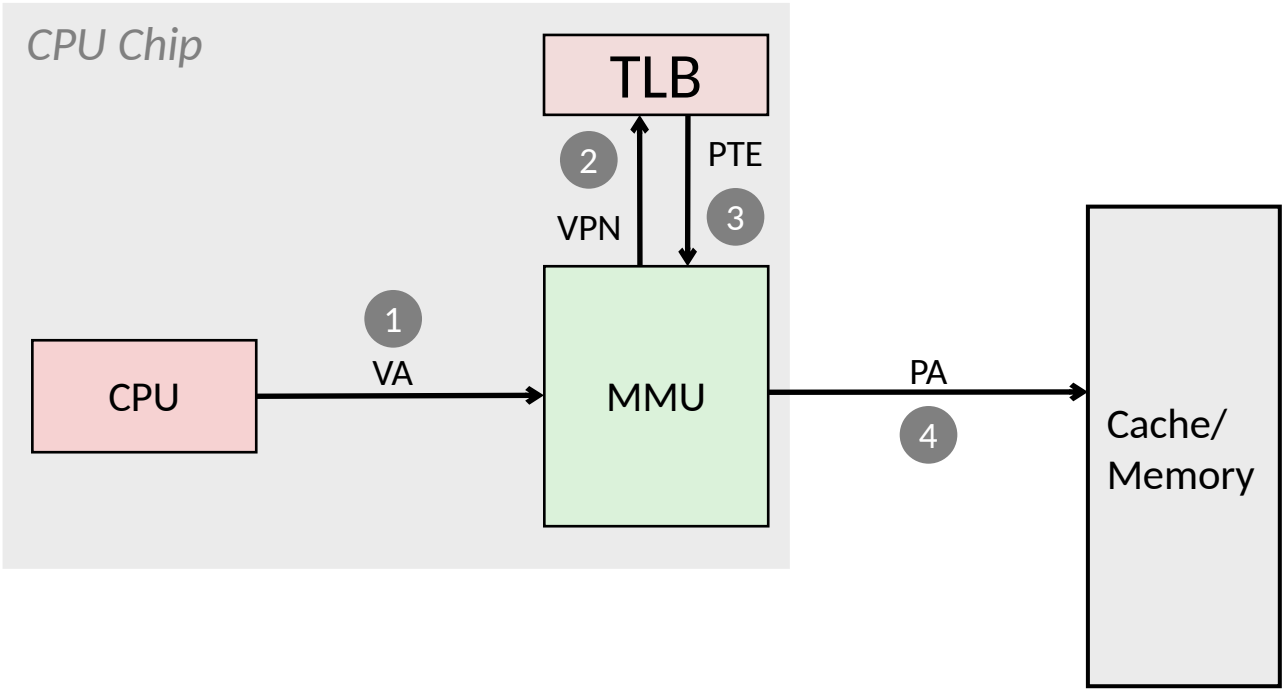
A TLB hit eliminates a memory access

TLB Hit



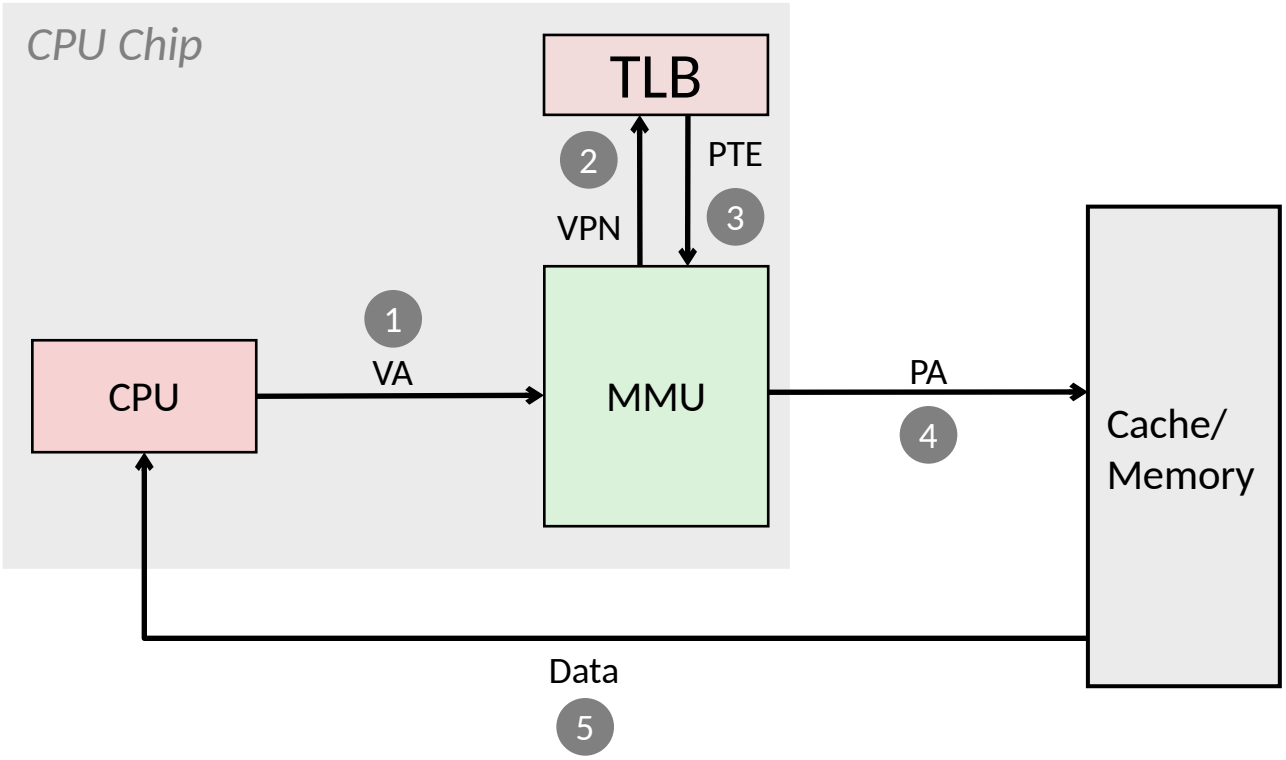
A TLB hit eliminates a memory access

TLB Hit



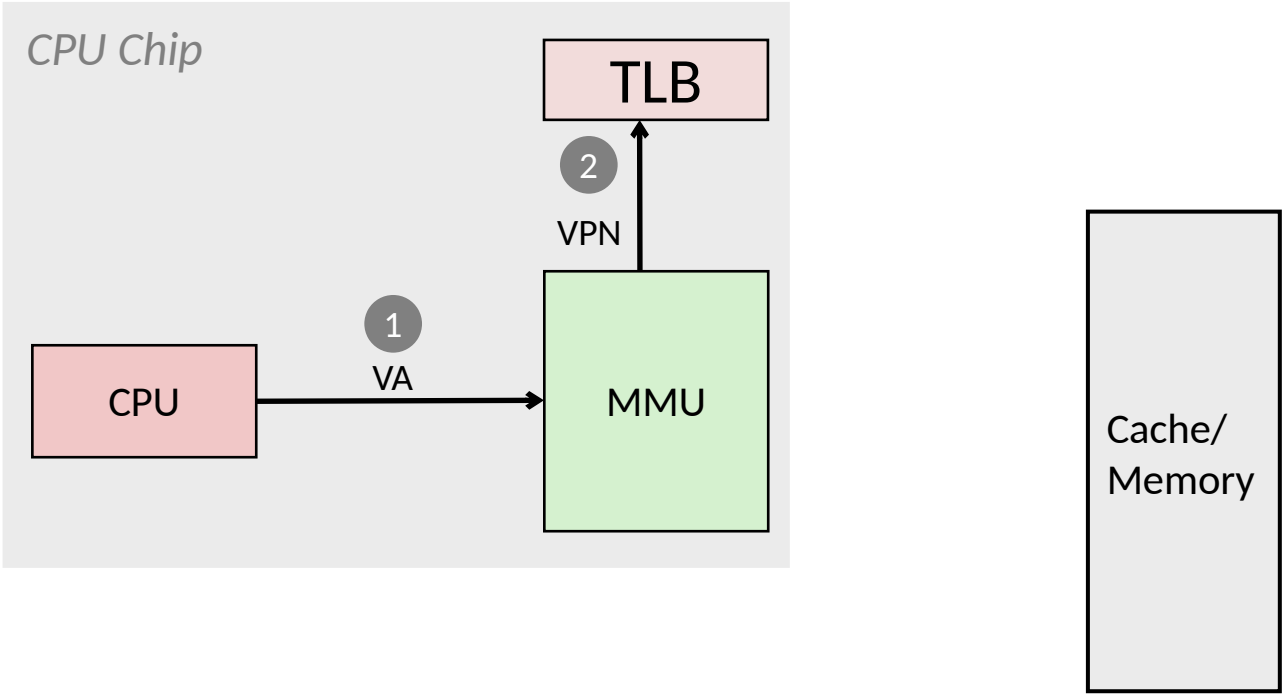
A TLB hit eliminates a memory access

TLB Hit



A TLB hit eliminates a memory access

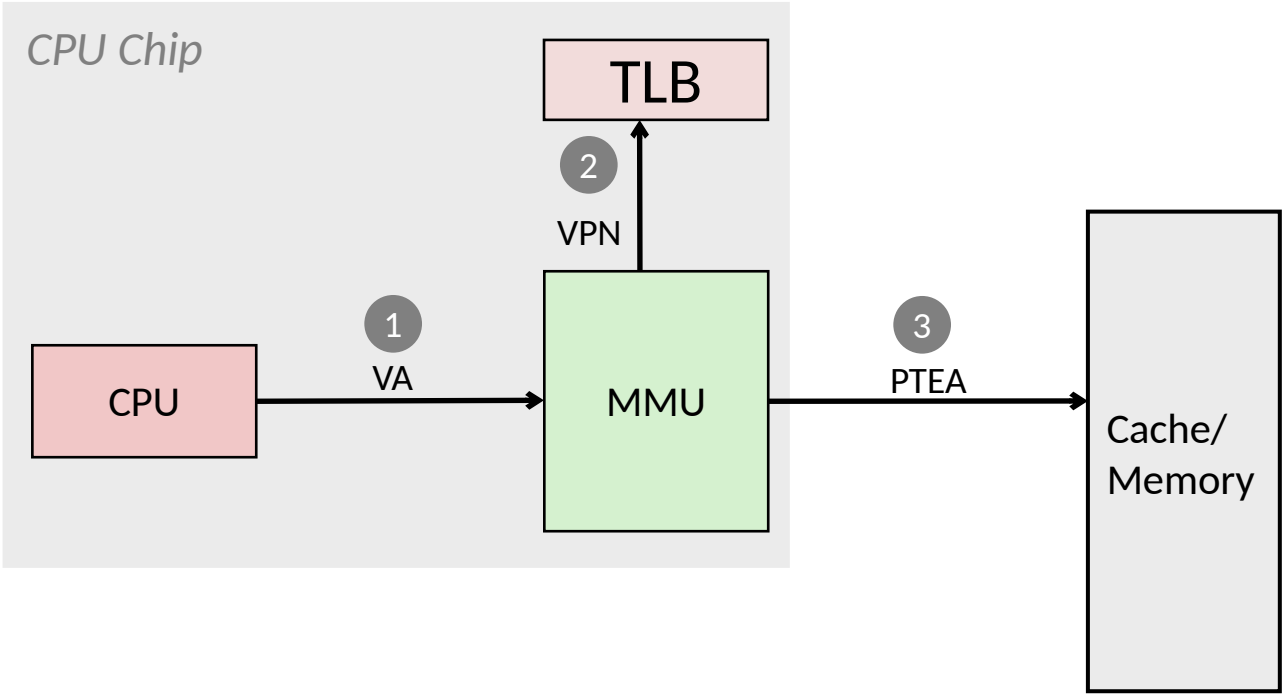
TLB Miss



A TLB miss incurs an additional memory access (the PTE)

Fortunately, TLB misses are rare. (Why?)

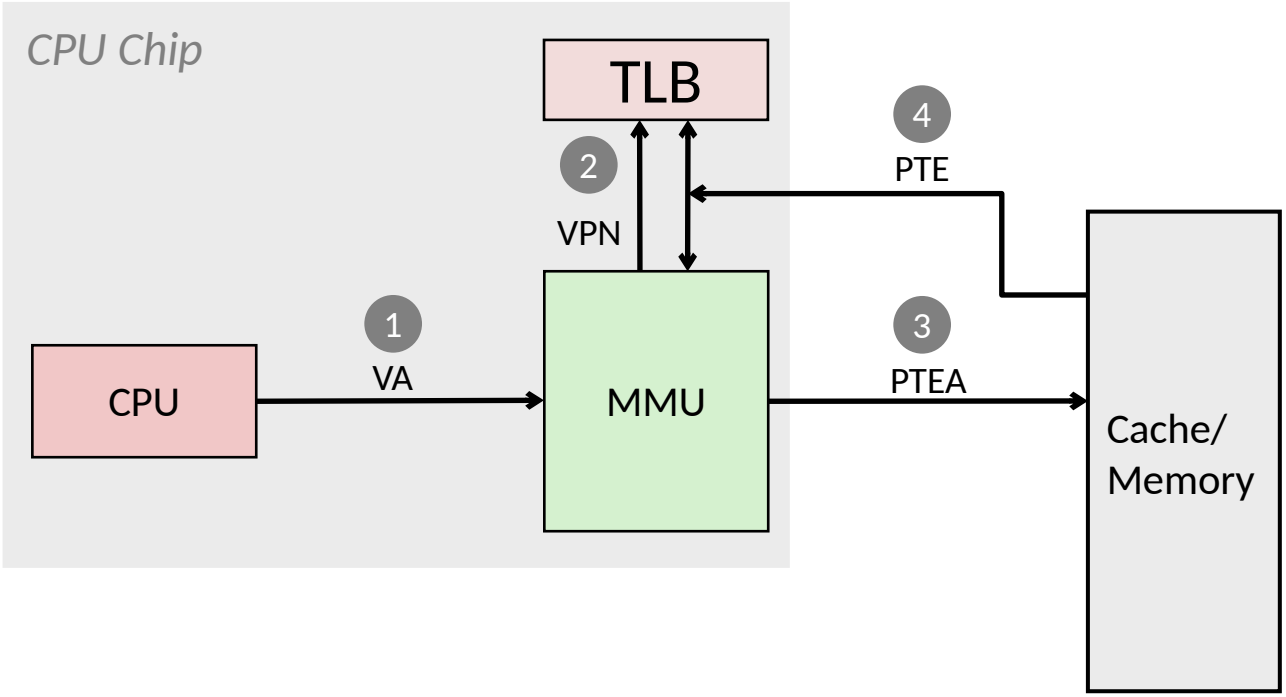
TLB Miss



A TLB miss incurs an additional memory access (the PTE)

Fortunately, TLB misses are rare. (Why?)

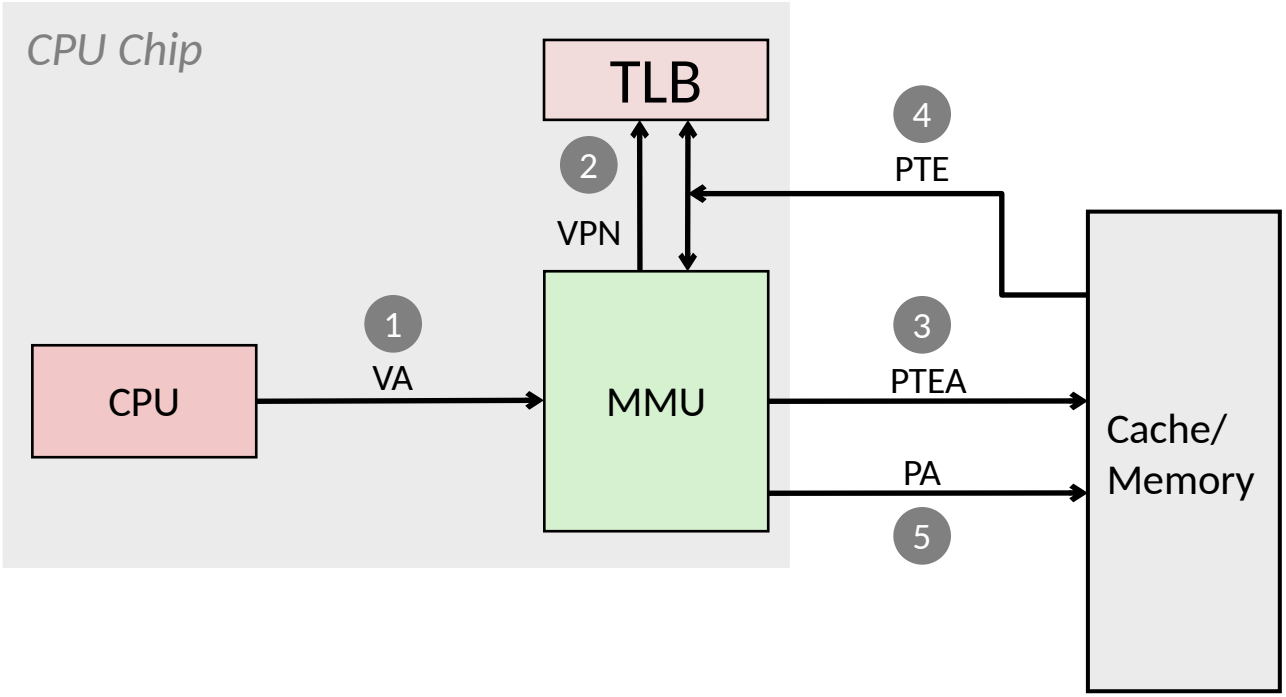
TLB Miss



A TLB miss incurs an additional memory access (the PTE)

Fortunately, TLB misses are rare. (Why?)

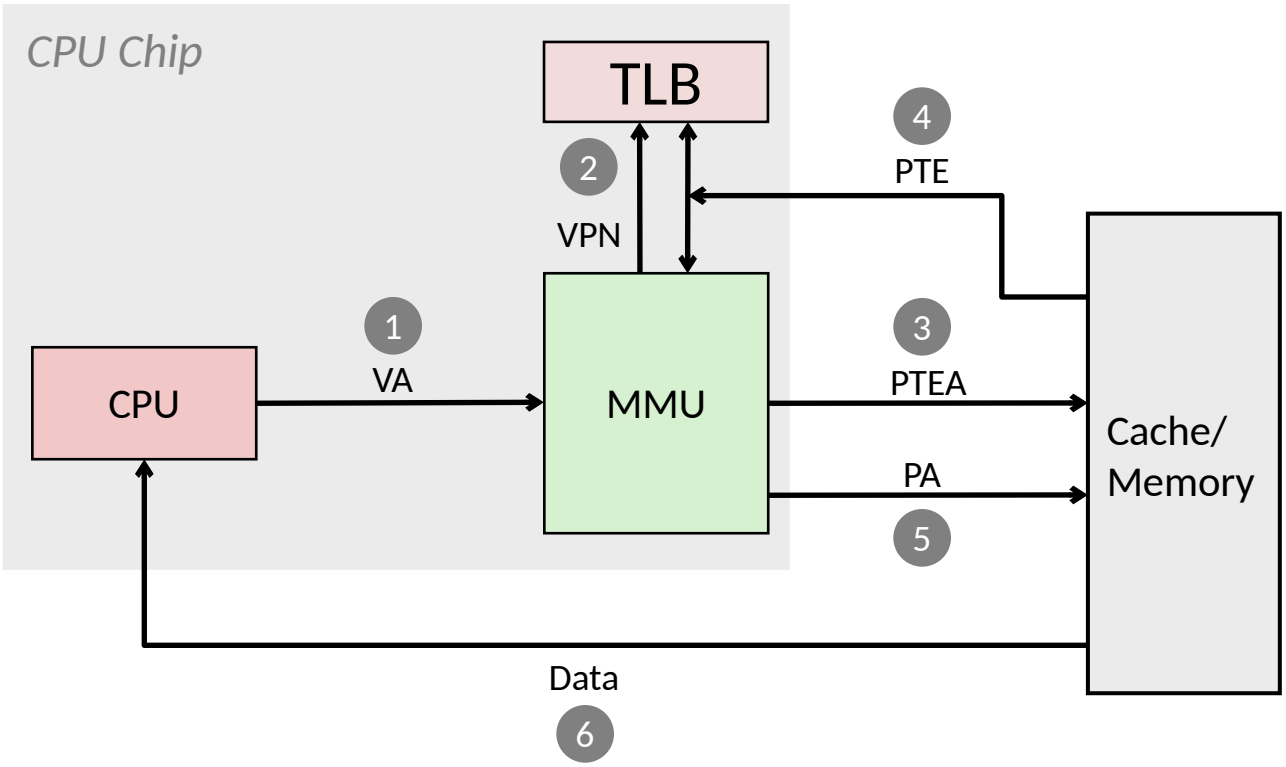
TLB Miss



A TLB miss incurs an additional memory access (the PTE)

Fortunately, TLB misses are rare. (Why?)

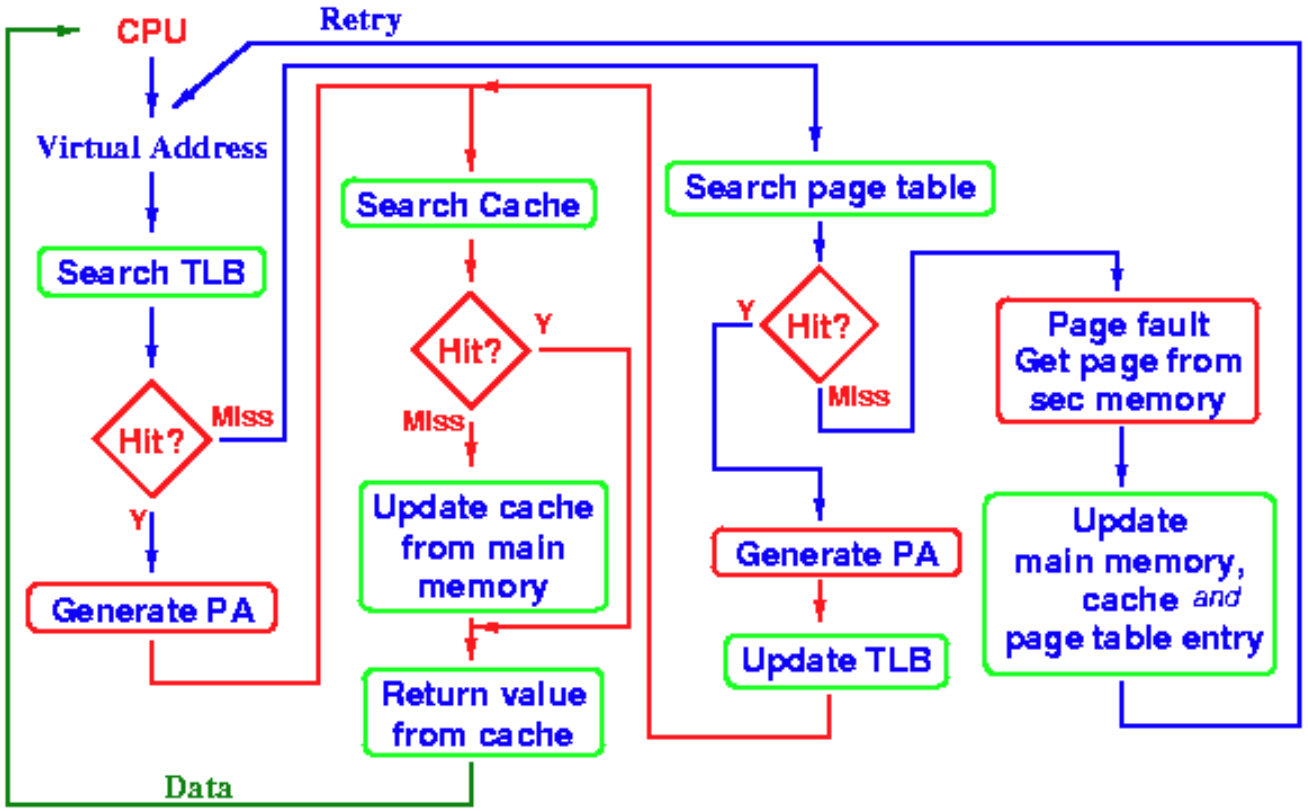
TLB Miss



A TLB miss incurs an additional memory access (the PTE)

Fortunately, TLB misses are rare. (Why?)

Integrating TLB and Cache



Discussion Questions

Your program repeatedly accesses a large array in memory, and suddenly performance drops despite a TLB hit rate over 95%. What's a likely reason?

A. The TLB is full, and cache misses are now handled by disk
B. The OS switches to segmentation when TLB is overloaded
C. The page table is misaligned and the MMU enters fallback mode
D. The array size exceeds the number of TLB entries, causing constant eviction and reloading of translations

Multi-Level Paging

- Problem: Storage overhead of page tables
  - 4KB (2^{12}) page size
  - 48-bit address space
  - 8-byte PTE
  - Page table size = $2^{48} * 2^{-12} * 2^{3} = 2^{39}$ bytes
  - 512 GB page table per process!

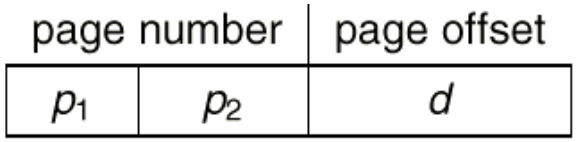
Multi-Level Paging

- Problem: Storage overhead of page tables
  - 4KB (2^{12}) page size
  - 48-bit address space
  - 8-byte PTE
  - Page table size = $2^{48} * 2^{-12} * 2^{3} = 2^{39}$ bytes
  - 512 GB page table per process!

- Solution: Multi-level paging
  - Breaks up virtual address space into multiple page tables in hierarchy
  - Maps a portion of the address space actually being used (tiny fraction of the entire address space)

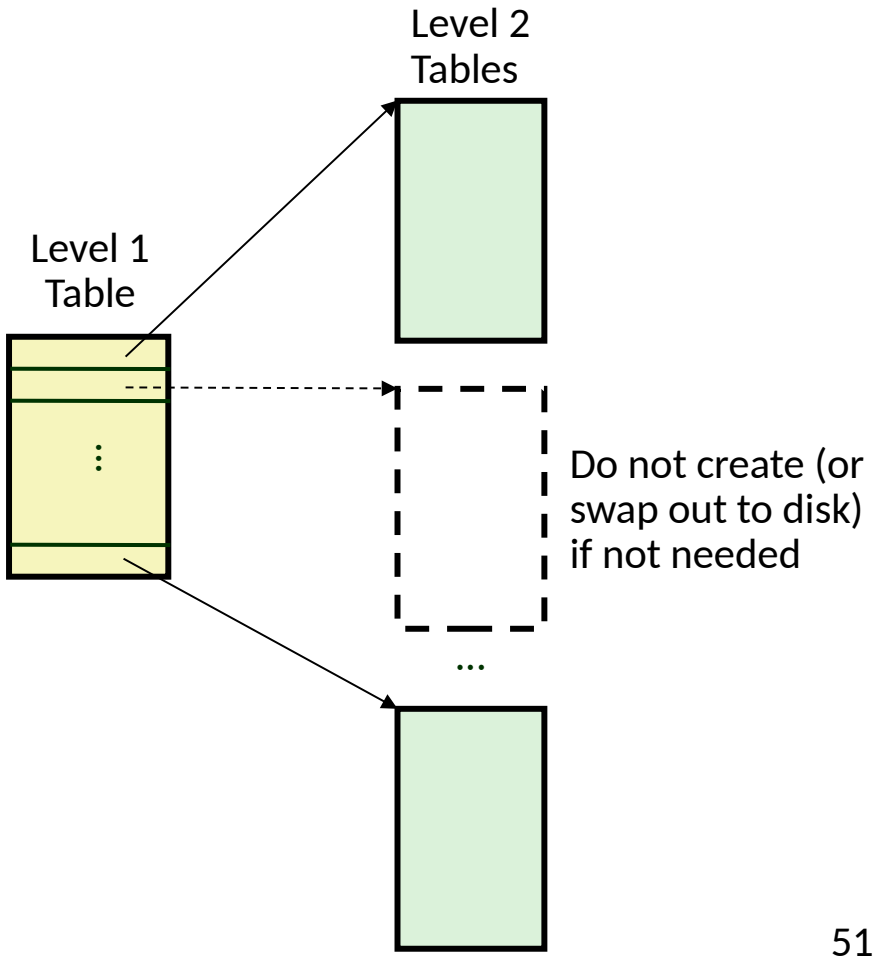
Multi-Level Page Tables

- Example: 2-level paging

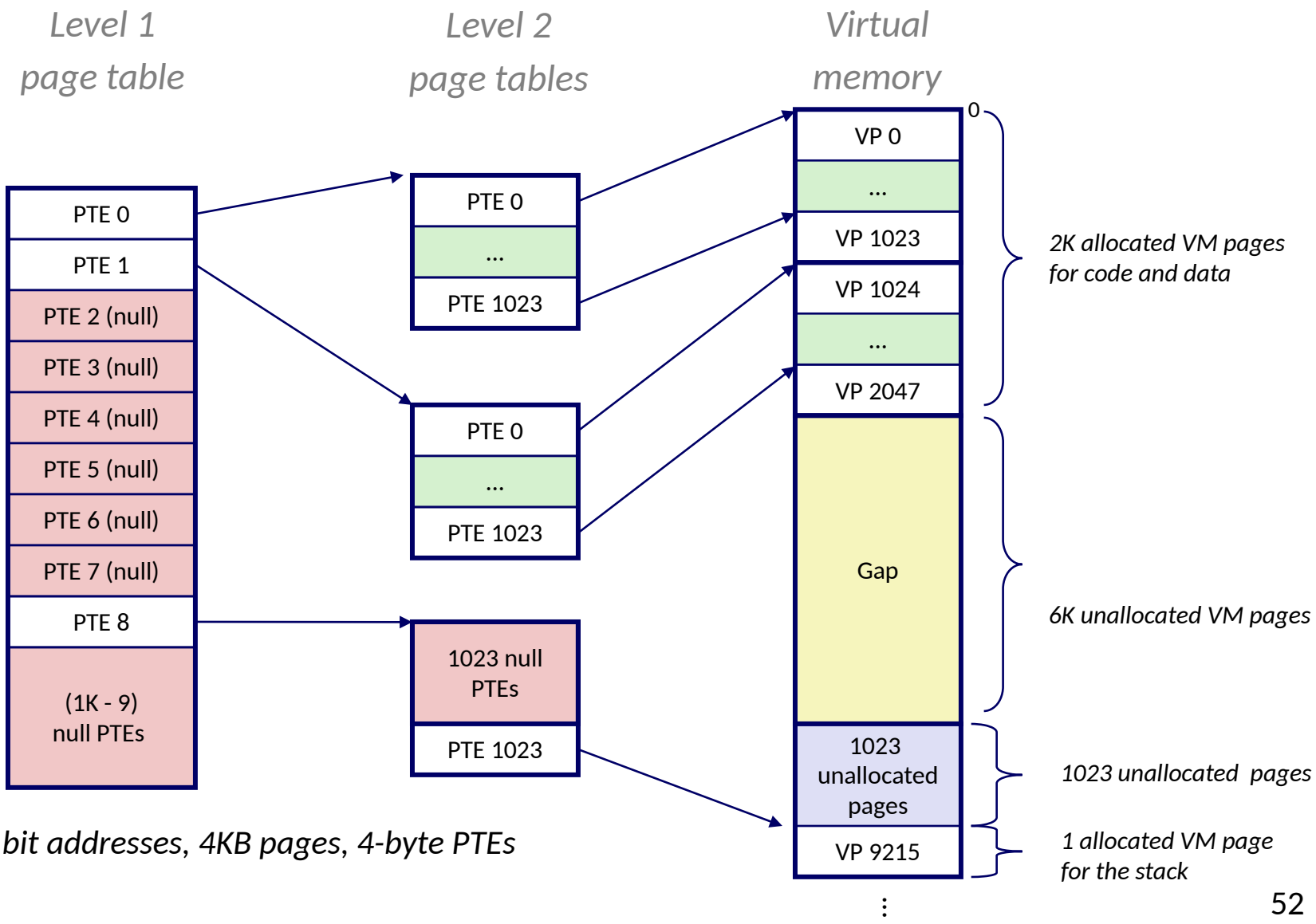


Level 1 table
- Each PTE points to a L2 page table
- Always memory resident

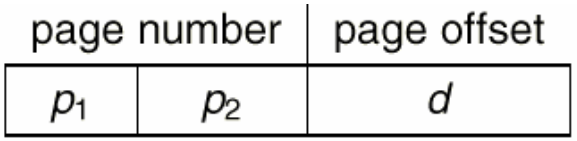
Level 2 table
- Each PTE points to a page
- Paged in and out like any other data



Two-Level Page Table Hierarchy



32 bit addresses, 4KB pages, 4-byte PTEs



- Consider two-level paging with a page size of 4 KB.
  - 32 bit address space
  - p1 is 10 bits
  - p2 is 10 bits
  - PTE size is 8 bytes

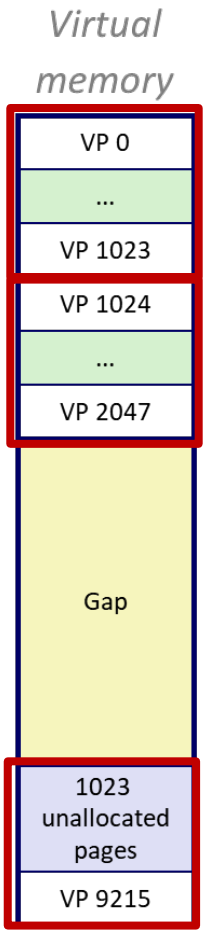
- How many PTEs in the Level-1 page table?

- How many Level-2 page tables can be created in the worst case?

page number		page offset
p_1	p_2	d

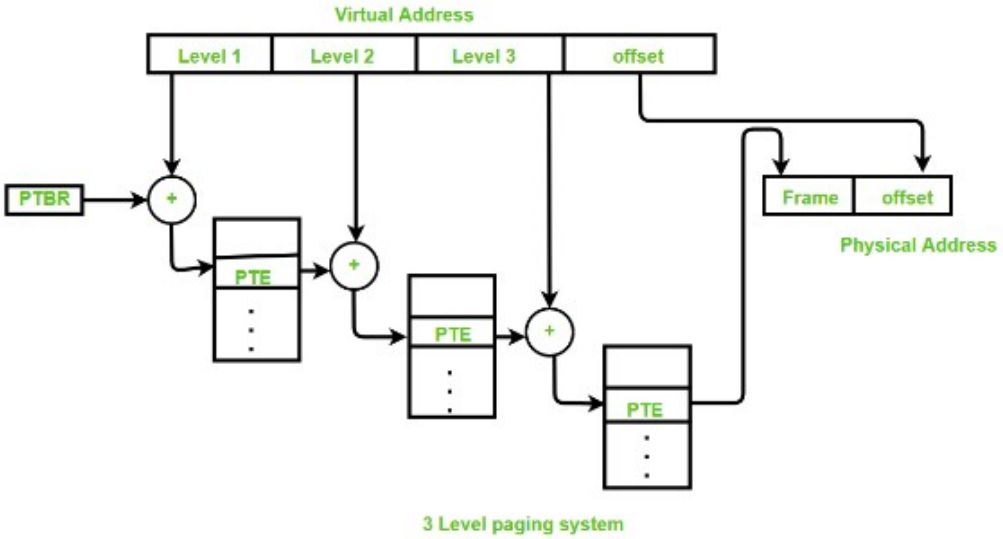
- Consider two-level paging with a page size of 4 KB.
  - 32 bit address space
  - p1 is 10 bits
  - p2 is 10 bits
  - PTE size is 8 bytes

- Suppose there are 3 memory regions in a virtual address space, each region can be covered by one Level-2 table.
  How much space is required for all page tables?



Multi-Level Paging

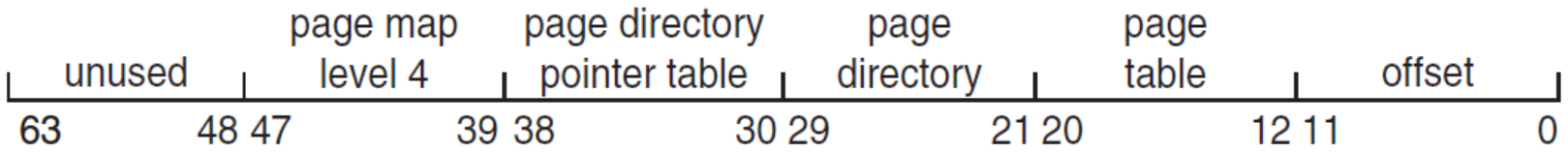
- Problem: Each (virtual) memory reference requires multiple memory references



- Trade off between spatial and temporal overhead
  - Memory access time, TLB hit ratio
  - Size of each page table

Intel x86-64 Paging

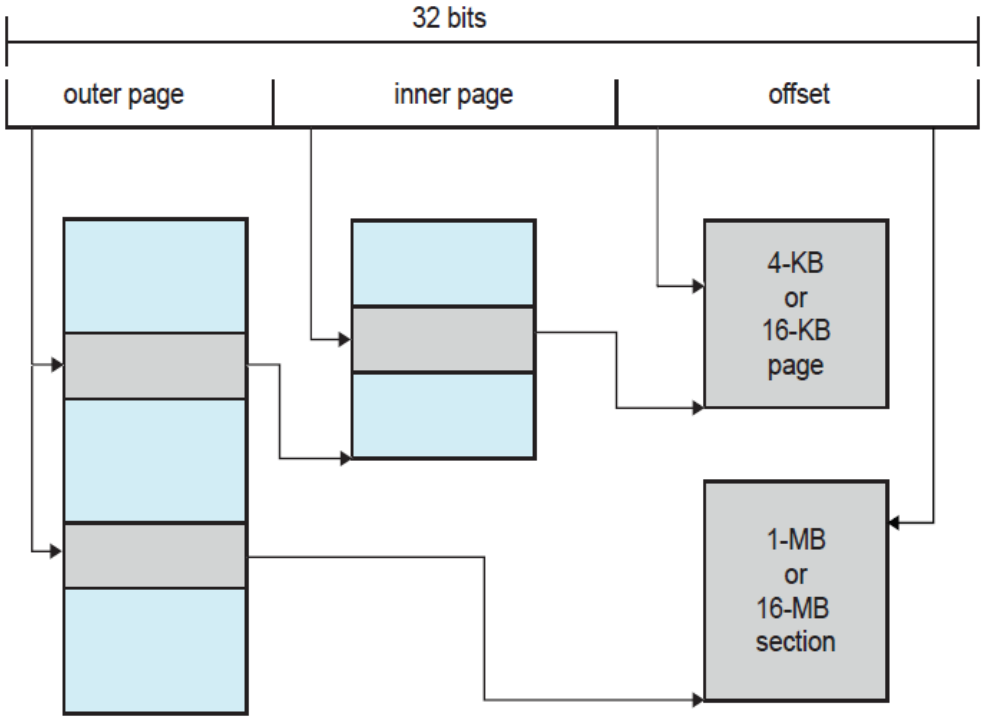
- Current generation Intel x86 CPUs
  - 64 bit architecture: maximum 2^{64} bytes address space
- In practice, only implement 48 bit addressing
  - Page sizes of 4 KB or 4 MB (optionally, 2 MB or 1 GB)
  - Four levels of paging hierarchy
    - Only use 48 bit addressing → 256 TiB of virtual address space



  - Recent processors have 5-level paging (e.g., Intel Ice Lake)
    - Use 57 bit addressing → 128 PiB of virtual memory space

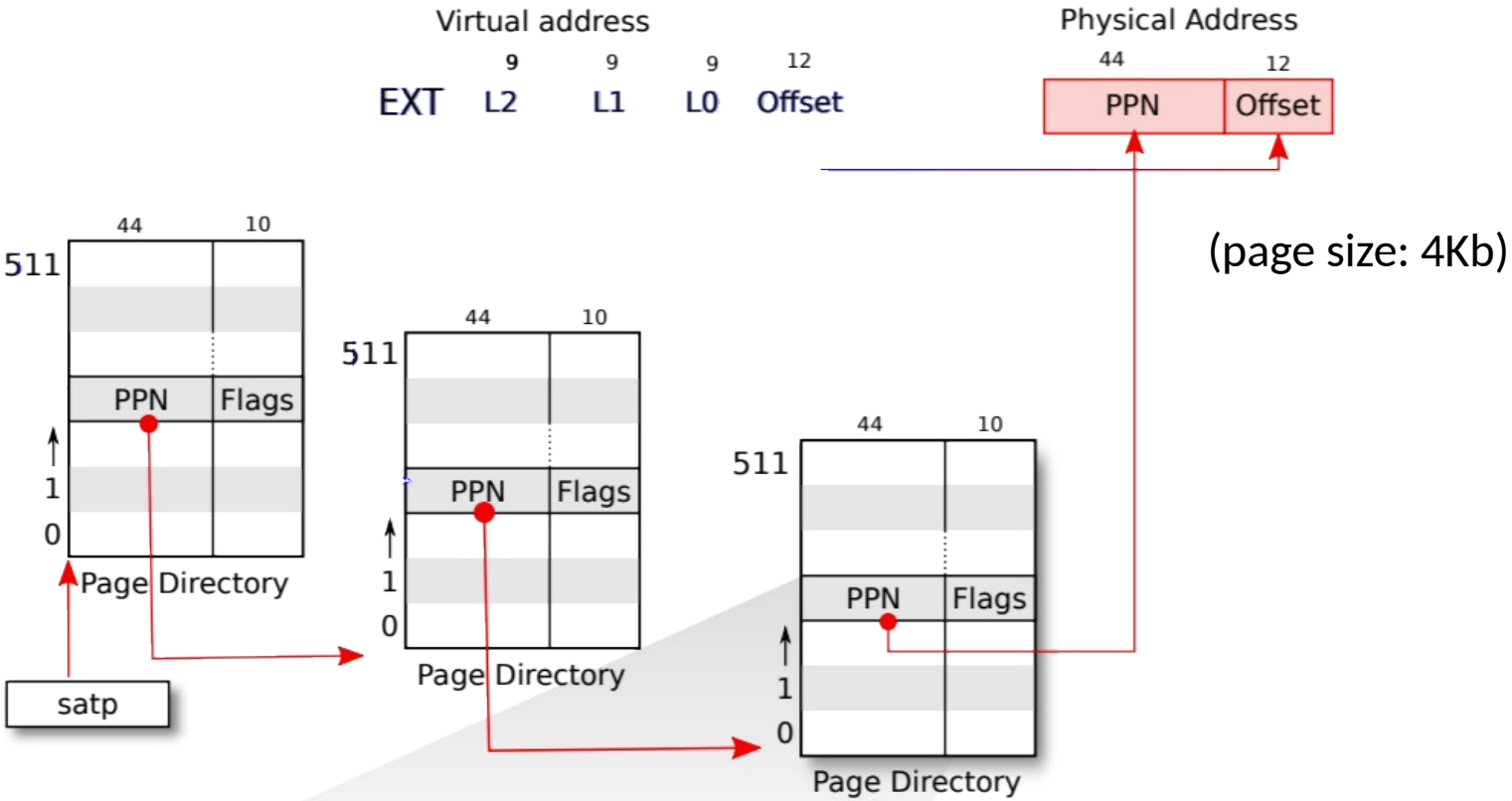
ARM Paging

- 4 KB, 16 KB, 1 MB pages
- 32 bit architecture
  - Up to two-level paging
- 64 bit architecture
  - Up to three-level paging



RISC-V Paging (xv6 runs on Sv39 RISC-V)

- Sv39: Page-based 39-bit virtual addressing
  - Bottom 39 bits of a 64-bit virtual address; the top 25 bits are not used
- Three-level paging



Discussion Questions

Multi-level page tables are widely used in modern systems. But under what realistic condition might a flat page table actually outperform them in terms of translation efficiency?

A. When the system frequently allocates and frees memory in random regions
B. When the virtual address space is large but sparsely used
C. When most of the virtual address space is valid and densely mapped
D. When page faults must be handled through user-space exception handlers

Discussion Questions

How do multi-level page tables and the TLB interact in modern systems, and under what condition can their interaction significantly impact memory access performance?

A. TLBs eliminate the need for page tables, so multi-level paging becomes irrelevant
B. Multi-level paging makes TLBs unnecessary if only the first level is cached
C. TLBs require flushing all levels of the page table on context switch
D. If a TLB miss occurs, multi-level paging can lead to multiple memory accesses for a single translation, increasing latency

Summary

- Virtual memory via paging provides several benefits, including the ability to run larger programs than the available physical memory, increased security through memory isolation, and simplified memory management for applications.
- However, using virtual memory also introduces additional overhead due to page table lookups and potential performance degradation if excessive swapping occurs.